МИНОБРНАУКИ РОССИИ

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ

**«САНКТ-ПЕТЕРБУРГСКИЙ ПОЛИТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ ПЕТРА ВЕЛИКОГО»**

Институт компьютерных наук и кибербезопасности

Высшая школа технологий искусственного интеллекта

Отчет по дисциплине «Управление знаниями и технологиями баз данных»

«Работа с метаданными и проектирование элементов BI системы»

Студент: _____ Салимли А. М.

Преподаватель: _____ Попов С. Г.

«27» 10. 2026 г.

Санкт-Петербург, 2026г.

# Содержание

**1 Постановка задачи** **4**

**2 Базы-источники** **5**
2.1 База данных: Учет репетиторов для школьников . . . . . 5
2.2 База данных: Всероссийские соревнования по горнолыжному спорту . . . . . 7
2.3 База данных: Спортсмены и соревнования в чирлидинге . . . . . 9

**3 Проектирование информационной схемы** **12**

**4 Проектирование базы данных шаблонов** **15**

**5 Контейнеризация** **16**
5.1 Тома . . . . . 16
5.2 Сети . . . . . 16
5.3 Порты . . . . . 17

**6 Проектирование архитектуры** **18**

**7 Разработка системы** **22**
7.1 Соединение . . . . . 22
7.2 Окружение . . . . . 22
7.3 Передача данных . . . . . 22
7.4 Шаблонный запрос . . . . . 23
7.5 Сохранение шаблонов запросов . . . . . 24
7.6 Перевод в NumPy массив . . . . . 25
7.7 Построение графиков . . . . . 25
7.8 Запуск . . . . . 25

**8 Интерфейс** **27**
8.1 Формы . . . . . 27
8.2 Выбор нескольких колонок . . . . . 27
8.3 Вывод таблиц . . . . . 27
8.4 Вывод графиков . . . . . 28
8.5 Вывод SQL запроса . . . . . 28
8.6 Форма условий . . . . . 28
8.7 Вывод отличий . . . . . 28
8.8 Шаблон интерфейса . . . . . 28

**9 Разработка интерфейса** **30**
9.1 Поля ввода . . . . . 30
9.2 Выпадающие списки . . . . . 30
9.3 Кнопки . . . . . 31
9.4 Выбор нескольких атрибутов . . . . . 31
9.5 Форма выбора шаблона . . . . . 31
9.6 Форма для условий . . . . . 32
9.7 Форма для сравнений . . . . . 32
9.8 Результат . . . . . 33

**10 Результаты работы** **35**
10.1 Сценарий 1 . . . . . 35
10.2 Сценарий 2 . . . . . 36
10.3 Сценарий 3 . . . . . 37
10.4 Сценарий 4 . . . . . 38

10.5 Сценарий 5 . . . . . 39
10.6 Сценарий 6 . . . . . 40
10.7 Сценарий 7 . . . . . 41
10.8 Сценарий 8 . . . . . 42
10.9 Сценарий 9 . . . . . 43
10.10 Сценарий 10 . . . . . 44
10.11 Сценарий 11 . . . . . 45

**Заключение** **47**

**Список источников** **48**

# 1 Постановка задачи

Требуется разработать систему, способную визуально показывать накопленные сведения в виде графиков и сводок, чтобы пользователь без специальной подготовки мог сам выбрать, какие данные анализировать, отображать или отбирать нужные по условиям, таким образом собирая отчет. Удачные варианты отчетов должны сохраняться для повторного использования. Система должна фиксировать, когда состав собираемых сведений изменился, и должна иметь возможность показывать эти изменения.

Отчет в данном случае подразумивает собой страницу, на которой отображен график по выбранным данным данным и сами данные, по условиям которые выбрал пользователь.

Система включает в себя три базы данных источников из единой СУБД работающих как отдельные независимые контейнеры с помощью Docker, отдельную независимую базу данных имеющую аналог информационной схемы, включающую в себя метаданные трех баз данных источников, а так же отдельную независимую базу данных содержащую шаблоны запросов в виде SQL запросов. Сами базы данных общаются между собой с помощью API. Разработать интерфейс, включающий в себя поля выбора конкретной базы данных источников из трех, выбор столбцов баз данных, отображение таблиц, фильтр «И», отображение SQL запроса, вывод графика столбчетой диаграммы. Кнопка просмотра отличий баз данных с возможностью обновления до текущего состояния.

Таким образом проектируется система работающая с метаданными из независимых баз данных, в единой СУБД, с минимальным интерфейсом BI системы.

Условия:

- СУБД: Postgres;
- Контейнеризация: Docker;
- Кол-во записей в базах данных источников: не менее 200000;
- Информационная схема: Денормализованная;
- Способ общения между базами: REST;
- Язык написания интерфейса: Python;
- Фильтрация по числовым данным для агрегированных функций;
- Библиотека отображения графиков: matplotlib, seaborn;
- Библиотека интерфейса: Streamlit;
- Правило фильтра: «И»;
- Запрос: Обычный вывод таблицы.

# 2 Базы-источники

## 2.1 База данных: Учет репетиторов для школьников

Предметная область - репетиторство, то есть индивидуальное или групповое обучение школьников, цель которого углубить знания по предмету, подготовиться к экзаменам или улучшить успеваемость. Репетитора, как правило, ищет родитель, когда понимает, что ребенку нужны дополнительные занятия. Занятия проходят в одном из форматов: очно, дистанционно (Zoom, Skype, Telegram и др.) или смешанно. После занятий родитель оставляет отзыв о репетиторе, а репетитор - отзыв об ученике. Цели базы данных: поиск репетитора, рейтинг и отзывы о репетиторах и учениках, оценка востребованности преподавателей. Данные сгенерированы: около 300 000 занятий, 40 000 родителей, 4 000 репетиторов и по 60 000 отзывов каждого вида.

Схема extra_for_students включает двенадцать таблиц. Таблица gender - справочник полов. Атрибуты: gender_id - идентификатор, gender_name - название. Таблица city - справочник городов. Атрибуты: city_id - идентификатор, city_name - название города, timezone - часовой пояс. Таблица format - формат занятия (очно, дистанционно, смешанный). Атрибуты: format_id - идентификатор, format_name - название формата. Таблица subjects - справочник школьных предметов. Атрибуты: subject_id - идентификатор, subject_name - название предмета.

Таблица parent хранит родителей учеников. Атрибуты: parent_id - идентификатор, last_name, first_name, middle_name - фамилия, имя и отчество, contact - контакт для связи. Таблица tutors хранит репетиторов. Атрибуты: tutor_id - идентификатор, last_name, first_name, middle_name - ФИО, gender_id - ссылка на пол, city_id - ссылка на город, age - возраст (от 18 до 80 лет), experience - опыт преподавания в годах, contact - контакт. Связи N:1 с gender и city. Таблица learner хранит учеников. Атрибуты: learner_id - идентификатор, parent_id - ссылка на родителя, city_id - ссылка на город, last_name, first_name, middle_name - ФИО, study_goal - цель занятий, class - класс (от 1 до 11). Связь N:1 с parent: у одного родителя может быть несколько детей.

Таблица tutor_subject реализует связь N:N между tutors и subjects (какие предметы преподает репетитор) через составной первичный ключ (tutor_id, subject_id). Таблица learner_subjects аналогично реализует связь N:N между learner и subjects (по каким предметам занимается ученик), ключ (learner_id, subject_id).

Таблица lesson - центральная таблица фактов, содержащая около 300 000 записей. Атрибуты: lesson_id - идентификатор занятия, subject_id - ссылка на предмет, format_id - ссылка на формат, learner_id - ссылка на ученика, tutor_id - ссылка на репетитора, lesson_date - дата занятия, cost - стоимость в рублях, duration - длительность в минутах. Таблица review_of_tutor хранит отзывы родителей о репетиторе своего ребенка. Атрибуты: review_id - идентификатор, learner_id, parent_id, tutor_id - ссылки на ученика, родителя и репетитора, review_date - дата отзыва, rating - оценка от 1 до 5, review_text - текст отзыва. Таблица review_of_learner хранит отзывы репетитора об ученике. Атрибуты: review_id - идентификатор, tutor_id, learner_id - ссылки на репетитора и ученика, review_date - дата, rating - оценка от 1 до 5, review_text - текст отзыва.

Ниже на рисунке 1, показана схема базы данных:

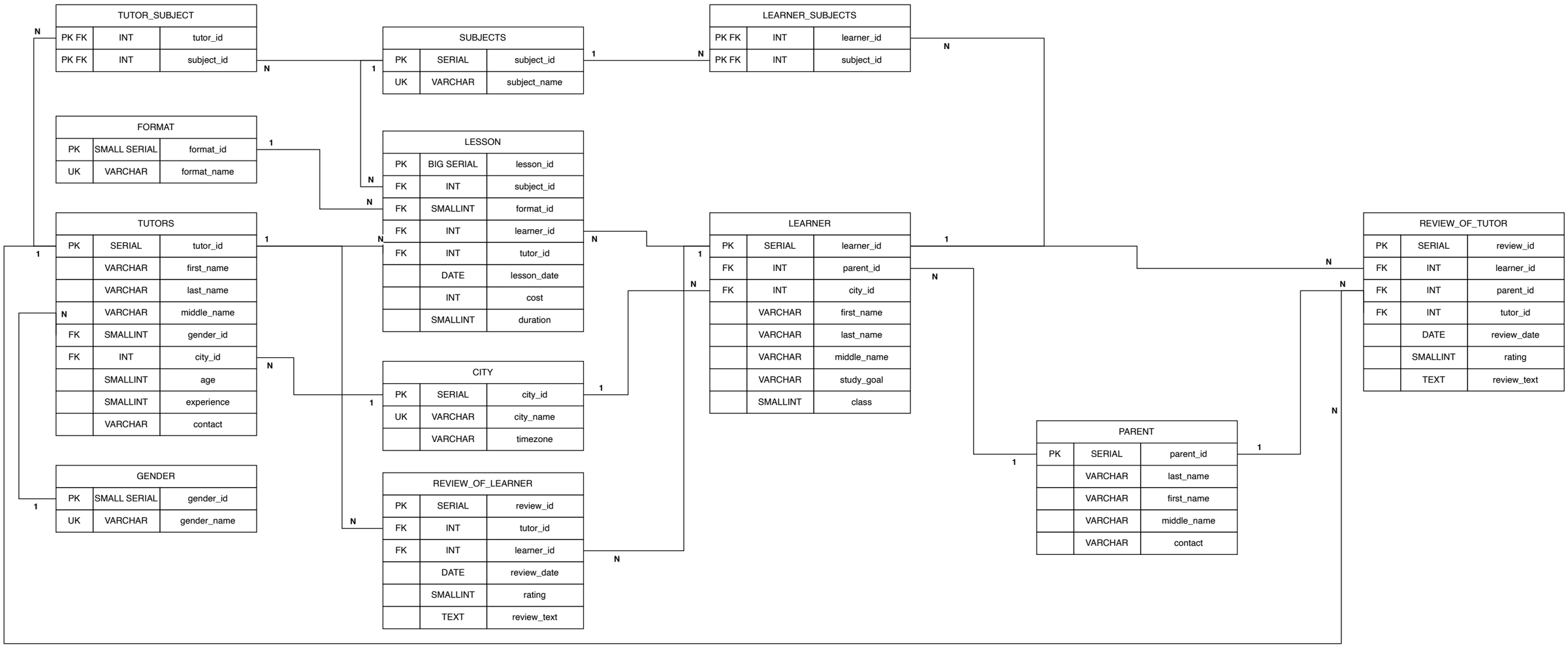



Рис. 1: Архитектура базы данных учета репетиторов

## 2.2 База данных: Всероссийские соревнования по горнолыжному спорту

Предметная область - всероссийские соревнования по горнолыжному спорту. Горнолыжный спорт включает пять дисциплин: слалом, гигантский слалом, супергигант, скоростной спуск и комбинацию. Спортсмены выступают за команды (сборные городов и регионов), старты проходят на горнолыжных курортах, за соблюдением правил следят судьи. В технических дисциплинах выполняется две попытки, во вторую проходят 30 лучших по первой, итоговое время складывается из двух попыток. Спортсмен может не финишировать (DNF), быть дисквалифицирован (DSQ) или не стартовать (DNS). Цели базы данных: личный и командный рейтинг за сезон, места участников, подбор места проведения по погодным условиям. Данные сгенерированы: 6 000 спортсменов, 4 500 стартов и около 190 000 результатов.

Схема mountain_sport включает восемь таблиц. Таблица team хранит команды. Атрибуты: team_id - идентификатор, team_name - название команды. Таблица athlete хранит спортсменов. Атрибуты: athlete_id - идентификатор, first_name, last_name - имя и фамилия, birth_date - дата рождения, gender - пол (M или F), team_id - ссылка на команду. Связь N:1 с team: в команде состоит несколько спортсменов. Таблица venue хранит горнолыжные курорты и трассы. Атрибуты: venue_id - идентификатор, venue_name - название, winter_weather_conditions и spring_weather_conditions - погодные условия зимой и весной. Таблица judge хранит главных судей соревнований. Атрибуты: judge_id - идентификатор, last_name, first_name - фамилия и имя.

Таблица competition хранит старты. Атрибуты: competition_id - идентификатор, competition_name - название, discipline - дисциплина, competition_date - дата, attempt1_result и attempt2_result - лучшее время первой и второй попытки на старте в секундах, venue_id - ссылка на место проведения, judge_id - ссылка на главного судью. Связи N:1 с venue и judge. Таблица athlete_competition реализует связь N:N между athlete и competition (заявка спортсмена на старт). Атрибуты: athlete_competition_id - идентификатор, athlete_id, competition_id - ссылки, на пару (athlete_id, competition_id) установлено уникальное ограничение. Таблица attempt - справочник итогов выступления. Атрибуты: attempt_id - идентификатор, attempt_status - статус (финишировал, DNF, DSQ, DNS).

Таблица result - центральная таблица фактов, содержащая около 190 000 записей. Атрибуты: result_id - идентификатор, athlete_id - ссылка на спортсмена, competition_id - ссылка на старт, attempt1_result и attempt2_result - время первой и второй попытки в секундах (NULL, если спортсмен не финишировал или не прошел во вторую попытку), attempt_id - ссылка на итог выступления. Спортсмен имеет не более одного результата на старте, что обеспечивается уникальным ограничением (athlete_id, competition_id).

Далее показана схема базы данных горнолыжных соревнований 2:

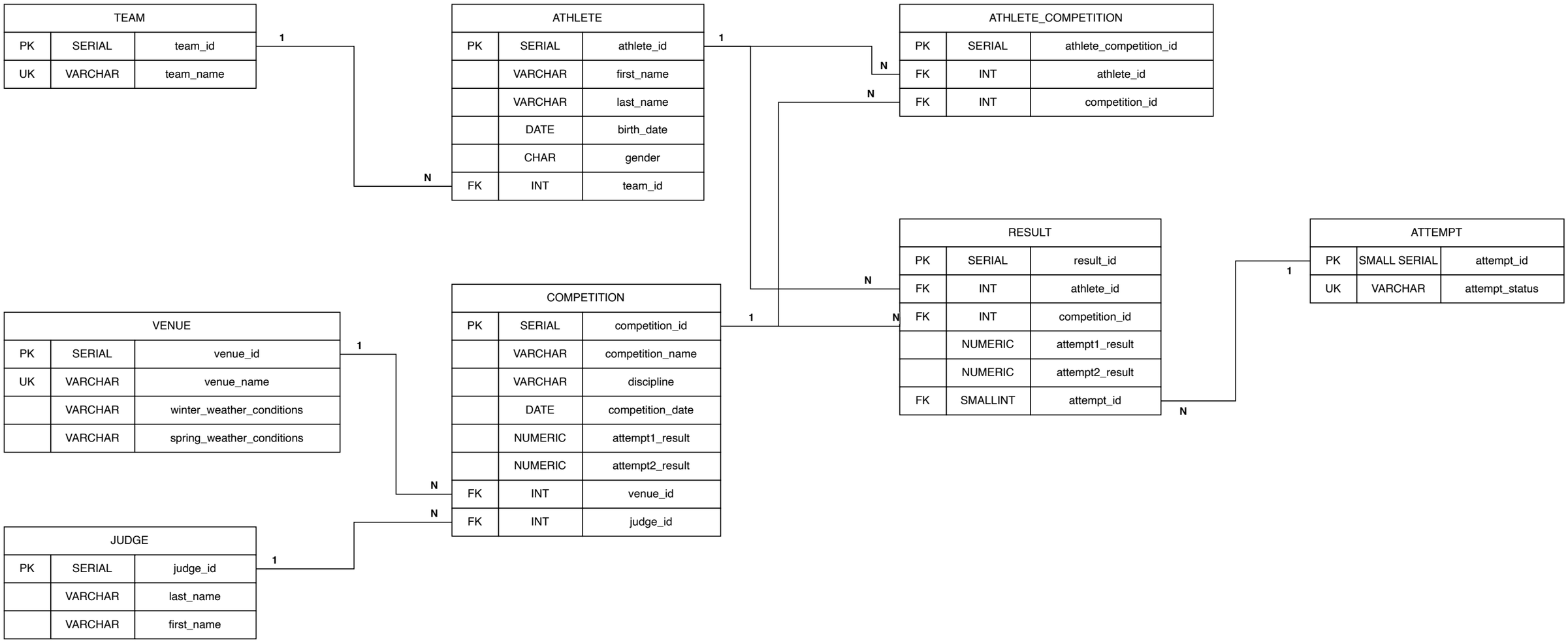



Рис. 2: Архитектура базы данных горнолыжных соревнований

### 2.3 База данных: Спортсмены и соревнования в чирлидинге

Предметная область - чир спорт, сочетающий элементы танцев, гимнастики и акробатики. Спортсмены состоят в клубах, клубы относятся к региональным федерациям, регионы к странам. Спортсмены выполняют разряды, от юношеских до КМС и мастера спорта, разряд подтверждается на официальных соревнованиях и оценивается судейской бригадой. Соревнования бывают разного уровня (городские, региональные, всероссийские, международные), выступления проходят в номинациях (чир-данс, чир-микс, фристайл, группа, двойка, соло) и возрастных категориях. Выступления оценивает судейская бригада, максимальный балл - 100, за нарушения выставляются сбавки, возможна дисквалификация. Цели базы данных: оценка выступлений судьями, учет спортсменов, определение лучших спортсменов, клубов и регионов, формирование цифрового паспорта спортсмена. Данные сгенерированы: 60 000 спортсменов, 1 200 клубов, 1 200 соревнований, 2 000 судей и не менее 300 000 записей участия спортсменов в выступлениях.

Схема sport_and_cheerleaders включает шестнадцать таблиц. Таблица country - справочник стран. Атрибуты: country_id - идентификатор, country - название страны. Таблица region хранит регионы. Атрибуты: region_id - идентификатор, region_name - название, region_ranking - место региона в рейтинге, sportsmans_count - количество спортсменов, country - ссылка на страну. Связь N:1 с country, пара (country, region_name) уникальна. Таблица club хранит клубы. Атрибуты: club_id - идентификатор, club_name - название, club_ranking - место в рейтинге, region_id - ссылка на регион, birthdate - дата основания, email - почта клуба.

Таблица grade_type - справочник разрядов и званий (юношеские, 1–3 разряд, КМС, МС, МСМК). Атрибуты: grade_type_id - идентификатор, grade_type - название. Таблица competition_type - справочник уровней соревнований. Атрибуты: competition_type_id - идентификатор, competition_type - название уровня. Таблица grade хранит приказы о присвоении разряда. Атрибуты: grade_id - идентификатор, grade_type - ссылка на разряд, date - дата присвоения, competition_type - ссылка на уровень соревнований, на котором выполнен разряд, verification - подтвержден ли разряд федерацией. Таблица sportsman хранит спортсменов. Атрибуты: sportsman_id - идентификатор, sportsman_name, sportsman_lastname, sportsman_middlename - имя, фамилия и отчество, grade_id - ссылка на приказ о разряде (NULL, если разряда нет), club_id - ссылка на клуб, sportsman_birthdate - дата рождения.

Таблица location - справочник мест проведения. Атрибуты: location_id - идентификатор, location - адрес площадки. Таблица competition хранит соревнования. Атрибуты: competition_id - идентификатор, competition_name - название, competition_date - дата, competition_type - ссылка на уровень, location - ссылка на место проведения. Таблица perfomance_type - справочник номинаций. Атрибуты: perfomance_type_id - идентификатор, perfomance_type - название номинации. Таблица age_category - справочник возрастных категорий. Атрибуты: age_category_id - идентификатор, age_category - название категории.

Таблица judge_type - справочник ролей судей (главный судья, судья по технике, по артистизму, по сложности). Атрибуты: judge_type_id - идентификатор, judge_type - название роли. Таблица judge хранит судей. Атрибуты: judge_id - идентификатор, judge_name, judge_lastname, judge_middlename - ФИО, judge_type - ссылка на роль, judge_score - квалификационный балл судьи (от 0 до 100). Таблица judge_panel хранит судейские бригады. Атрибуты: judge_panel_id - идентификатор, judge_count - число судей, disqualification - была ли вынесена дисквалификация, total_score - сумма выставленных баллов, total_allowance - сумма сбавок. Таблица judge_panel_judge реализует связь N:N между judge_panel и judge (состав бригады). Атрибуты: judge_panel_judge_id - идентификатор, judge_panel_id, judge_id - ссылки.

Таблица perfomance хранит выступления. Атрибуты: perfomance_id - идентификатор, competition_id - ссылка на соревнование, perfomance_place - занятое место (NULL при дисквалификации), perfomance_type - ссылка на номинацию, sportsman_count - число спортсменов в составе, age_category - ссылка на возрастную категорию, judge_panel - ссылка на судейскую бригаду. Таблица sportsman_perfomance - центральная таблица фактов, реализующая связь N:N

между sportsman и perfomance (состав выступления), содержит не менее 300 000 записей. Атрибуты: perfomance_sportsman_id - идентификатор, perfomance_id, sportsman_id - ссылки, пара (perfomance_id, sportsman_id) уникальна.

На рисунке 3 показана схема базы данных чирлидинга:

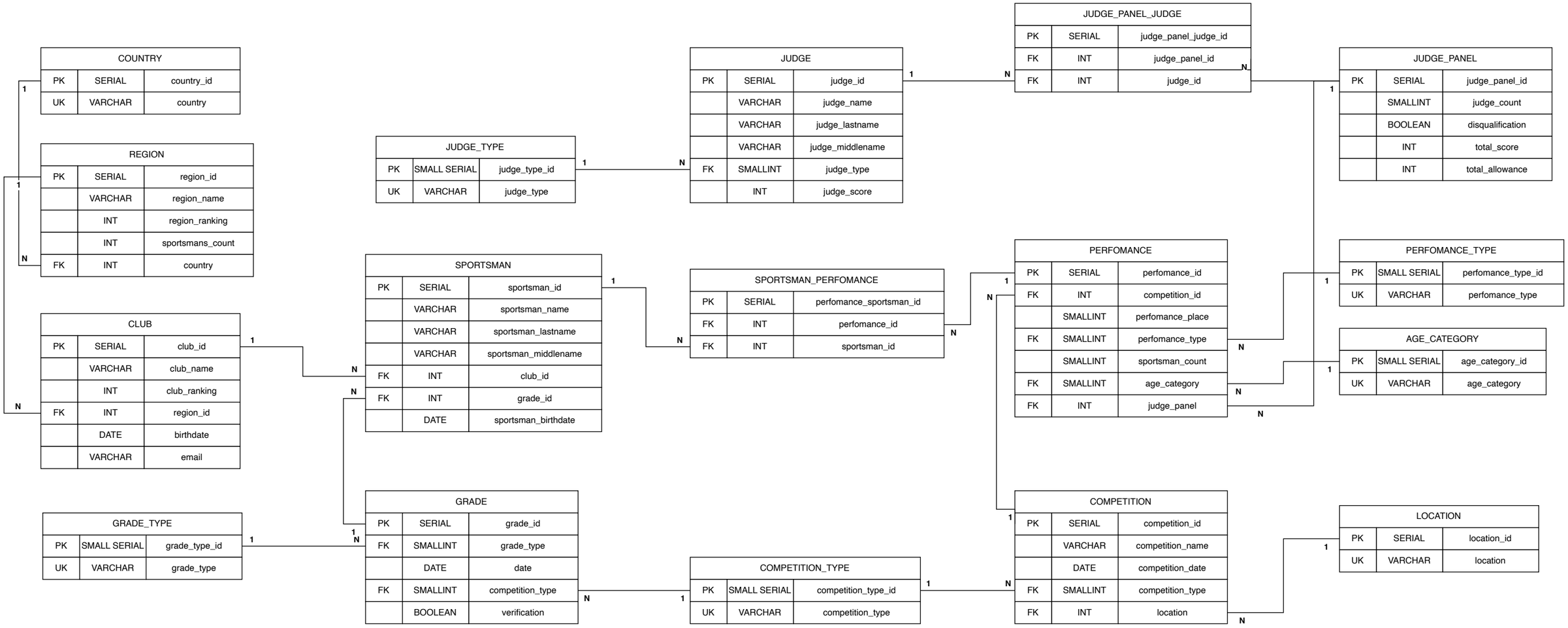



Рис. 3: Архитектура базы данных спортсменов и соревнований в чирлидинге

# 3 Проектирование информационной схемы

На рисунке 4, представлена схема базы данных имплементирующую логику денормализованной информационной схемы.

| META_CATALOG | | |
|---|---|---|
| PK1, FK1 | VARCHAR | source_id |
| PK2 | VARCHAR | schema_name |
| PK3 | VARCHAR | table_name |
| PK4 | VARCHAR | column_name |
| | VARCHAR | api_url |
| | INTEGER | ordinal_position |
| | VARCHAR | data_type |
| | VARCHAR | pk_key |
| | VARCHAR | fk_key |
| FK1 | VARCHAR | ref_schema_name |
| FK1 | VARCHAR | ref_table_name |
| FK1 | VARCHAR | ref_column_name |
| | TIMESTAMP | synced_at |

Рис. 4: Архитектура базы метаданных

Тут, таблица meta.catalog - это аналог информационной схемы, в которой каждая строка это одна колонка источника. Это было сделано для того, что бы в последующих шаблонных запросах не использовать JOIN механизм: вся информация о колонке, ее таблице, источнике и связях лежит в одной строке. Таблица разделена на несколько уровней.

Уровень источника:

- source_id - идентификатор источника (одна из трех баз). Нужен, что бы отличать одинаковые по имени таблицы из разных источников, и по нему же строится ссылка на meta.sources, где хранятся адреса баз данных источников;
- api_url - URL адрес источника. Хранится прямо в строке каталога, что бы по выбранной колонке сразу знать, к какому агенту отправлять запрос, без обращения к другой таблице.

Уровень таблицы:

- schema_name - имя схемы в БД источника. Нужно, так как в разных схемах могут быть таблицы с одинаковым именем;
- table_name - имя таблицы в БД источника. Вместе с schema_name используется в блоке FROM генерируемого SQL-запроса.

Уровень колонки:

- column_name - имя самой колонки. Используется в SELECT, WHERE, GROUP BY генерируемого запроса и как подпись в интерфейсе;
- ordinal_position - порядковый номер колонки в исходной таблице. Позволяет выводить колонки в интерфейсе в том же порядке, что и в источнике, а так же участвует в сравнении структур (Show diff): если порядок колонок изменился, это будет видно как отличие;

- data_type - тип данных колонки в источнике. Тип определяет, что можно делать с колонкой: числовые колонки подходят для агрегатных функций (SUM, AVG) и оси значений графика, даты и время для оси времени, строковые для группировки и фильтров с текстовыми операторами. Так же тип сравнивается при Show diff.

Уровень ключей (описание связей):

- pk_key - признак первичного ключа. Значение PK ставится, если ключ таблицы одиночный. Если ключ составной, то колонки нумеруются: PK1, PK2, ... Номер это позиция колонки в составном ключе. Он нужен, что бы понимать, какие колонки вместе однозначно определяют строку и в каком порядке идут. Если NULL, колонка не входит в первичный ключ;
- fk_key - номер внешнего ключа в таблице: FK1, FK2, ... Номер отличает один внешний ключ таблицы от другого: если таблица ссылается на две разные таблицы, то это будут FK1 и FK2. Колонки составного внешнего ключа получают один и тот же номер, по нему видно, что их нужно соединять вместе. Если NULL, колонка не является внешним ключом;
- ref_schema_name, ref_table_name, ref_column_name - на какую схему, таблицу и колонку ссылается внешний ключ. Заполняются только вместе с fk_key (это проверяется ограничением CHECK). По ним система находит, какие таблицы можно соединять между собой, и строит JOIN в запросе автоматически, а не заставляет пользователя выбирать его вручную.

Служебное поле: synced_at - время последней синхронизации строки с источником. Показывает, насколько актуален каталог, и обновляется при синхронизации, если структура таблицы в источнике отличается от сохраненной.

Первичным ключом таблицы является составной ключ (source_id, schema_name, table_name, column_name), так как этого набора достаточно, что бы однозначно определить колонку среди всех источников. Внешний ключ ссылается на саму таблицу (ref_* колонки указывают на строку каталога с тем же source_id), поэтому связи между таблицами хранятся внутри каталога.

Еще нужны ограничения таблицы, что бы каталог оставался согласованным:

- pk_catalog - составной первичный ключ, не дает добавить одну и ту же колонку источника дважды при синхронизации;
- fk_catalog_ref - внешний ключ на саму таблицу, гарантирует, что ссылка ref_* указывает на реально существующую колонку каталога. ON DELETE CASCADE удаляет связи вместе с удаленной колонкой, что бы не оставалось ссылок в никуда.
- ck_catalog_pk_key и ck_catalog_fk_key - проверяют формат значений (PK, PK1, PK2, ... и FK1, FK2, ...), что бы по ним можно было разбирать номер ключа;
- ck_catalog_fk_ref - поля fk_key и ref_* должны быть либо все пустыми, либо все заполненными. Это исключает «неполный»внешний ключ, по которому нельзя построить JOIN.

Ниже показан пример составного ключа: Таблица tutor_subject базы учета репетиторов, у которой первичный ключ (tutor_id, subject_id) и обе колонки одновременно ссылаются на tutors и subjects.

В meta.catalog та же информация лежит в двух строках, по одной на колонку (см. табл. 1). Строку определяет составной ключ каталога (source_id, schema_name, table_name, column_name), а составной ключ источника виден по номерам PK1 и PK2. Колонки schema_name и ref_schema_name в обеих строках равны extra_for_students.

Таблица 1: Пример составного ключа в meta.catalog для tutor_subject

| source_id | table_name | column_name | pk_key | fk_key | ref_table_name | ref_column_name |
|---|---|---|---|---|---|---|
| extra_for_students | tutor_subject | tutor_id | PK1 | FK1 | tutors | tutor_id |
| extra_for_students | tutor_subject | subject_id | PK2 | FK2 | subjects | subject_id |

Номер PK берется из ordinal_position первичного ключа, номер FK присваивается по порядку колонок в таблице. Колонки ref_* строки tutor_id указывают на строку каталога (extra_for_students, extra_for_students, tutors, tutor_id), то есть внешний ключ каталога ведет на саму таблицу каталога, и для построения достаточно одного запроса к базе метаданных.

# 4 Проектирование базы данных шаблонов

База данных предназначена для хранения шаблонов SQL-запросов, используемых в пользовательском интерфейсе системы. Схема содержит единственную таблицу form_templates, реализующую денормализованную структуру для хранения готовых запросов и параметров формы визуализации.

Атрибуты таблицы:

- template_id - первичный ключ таблицы;
- template_name - уникальное имя шаблона, что бы в интерфейсе можно было назвать или выбрать;
- sql_text - сам SQL запрос;
- source_id - идентификатор базы данных источника;
- schema_name, table_name - схема и таблица в базе источнике;
- chart_type - вид графика;
- dimension_col - имя категориальной переменой что б понимать какую колонку выбирать;
- measure_col - имя значений, так же что б понимать какую колонку выбрать;
- second_dim_col - вторая категориальная колонка для heatmap;
- agg_func - выбор аггрегатной функции (SUM, AVG, COUNT);
- filter_summary - читаемая сводка условий WHERE;
- row_limit - ограничение на количество строк (для LIMIT);
- created_at - дата и время создания шаблона;
- used_count - сколько раз шаблон применяли;
- last_used_at - дата последнего использования, так как планируется удалять шаблон если не используется более 10 дней.

Денормализованная структура таблицы позволяет восстанавливать параметры формы интерфейса из сохраненного шаблона без дополнительных соединений.

Сама схема преведена в рисунке 5:

| FORM_TEMPLATES | | |
|---|---|---|
| PK | BIGSERIAL | template_id |
| UK | VARCHAR | template_name |
| | TEXT NOT NULL | sql_text |
| | VARCHAR | schema_name |
| | VARCHAR | source_id |
| | VARCHAR | schema_name |
| | INT | table_name |
| | VARCHAR NOT NULL | dimension_col |
| | VARCHAR | measure_col |
| | VARCHAR | second_dim_col |
| | VARCHAR | agg_func |
| | VARCHAR | chart_type |
| | INT | row_limit |
| | TIMESTAMP | created_at |
| | TIMESTAMP | last_used_at |

Рис. 5: Архитектура базы шаблонов

# 5 Контейнеризация

Для развертывания системы используется Docker, которая даст изоляцию компонентов приложения. То есть позволит сделать все базы независимыми. Каждая база данных запускается в отдельном контейнере на основе образа postgres16 Alpine.

В составе системы пять независимых контейнеров postgres:

- extra_db - база данных учета репетиторов для школьников;
- mountain_db - база данных соревнований по горнолыжному спорту;
- sport_db - база данных спортсменов и соревнований в чирлидинге;
- meta_db - база данных метаданных;
- templates_db - база данных шаблонов SQL запросов.

Каждый контейнер представляет собой самостоятельный экземпляр postgres со своей базой данных, пользователем, паролем, каталогом хранения и сетевой конфигурацией. Такое разделение препятствует непосредственному объединению баз данных на уровне СУБД и обеспечивает независимость источников. Взаимодействие между компонентами системы выполняется с помощью подключения одной базы данных к другой через REST API.

Конфигурация каждого контейнера задается с помощью файла Docker Compose. Для всех экземпляров postgres используются следующие параметры:

- image - образ postgres версии 16 на базе Alpine Linux;
- container_name - фиксированное имя контейнера;
- restart: unless-stopped - автоматический перезапуск контейнера после сбоя или перезапуска Docker;
- environment - параметры инициализации базы данных;
- volumes - подключение постоянного хранилища и каталога начальной инициализации;
- networks - подключение контейнера к выделенной изолированной сети;
- healthcheck - проверка доступности postgres.

Параметр postgres_DB - задает имя базы данных при первом запуске контейнера, (postgres_USER, postgres_PASSWORD) - учетная запись пользователя. Для каждой БД создается отдельное значение (например для БД репетиторов - extra_user, extra_pass).

## 5.1 Тома

Для сохранения данных postgres используются именованные Docker-тома. Каждая база данных имеет собственный том, не используемый другими контейнерами. Каждый том подключается к каталогу /var/lib/postgres/data соответствующего контейнера. Фактическое размещение кластера postgres определяется параметром PGDATA, имеющим значение /var/lib/postgres/data/pgdata.

Использование отдельных именованных томов обеспечивает сохранность данных при удалении или пересоздании контейнеров.

## 5.2 Сети

Для каждой базы данных создается отдельная изолированная сеть Docker. В каждую сеть подключается только один контейнер postgres. То есть базы данных источников не имеют прямого сетевого доступа друг к другу и не могут выполнять непосредственные межбазовые SQL-запросы.

Обмен данными между базами данных, базой метаданных, базой шаблонов и пользовательским интерфейсом осуществляется через REST API. Сетевая изоляция обеспечивает независимость источников и ограничивает прямое взаимодействие между контейнерами.

### 5.3 Порты

Внутри всех контейнеров postgres используется стандартный порт: 5432. Для доступа с хостовой системы каждому контейнеру назначается отдельный внешний порт:

Таблица 2: Распределение портов

| Контейнер | Внутренний порт | Внешний порт |
|---|---|---|
| `templates_db` | 5432 | 5436 |
| `meta_db` | 5432 | 5437 |
| `extra_db` | 5432 | 5438 |
| `mountain_db` | 5432 | 5439 |
| `sport_db` | 5432 | 5440 |

Проброс портов выполняется по схеме:

$$ExternalPort : 5432$$

То есть например 5438:5432 означает, что подключения к порту 5438 хостовой системы перенаправляются на порт 5432 postgres внутри контейнера базы источника.

# 6 Проектирование архитектуры

Каждая база данных работает как отдельный контейнер, где каждый контейнер, имеет собственный том, собственную сеть для общения. В верхней части архитектуры показано, что запрос на получение метаданных идет по REST API, после чего метаданные записываются в главную базу данных (metadata). Справа от главной базы данных, база шаблонных запросов, которая так же общается по REST.

Интерфейс работает локально без контейнера. В создании интерфейса используются 4 библиотеки языка Python. Для создания веб приложения с интерфейсом используется streamlit, для отображения графиков matplotlib и seaborn, а для чтения данных и перевода их в DataFrame и в послествии в numpy массив для отображения графиков - pandas.

Ниже на рисунке 6, предствлена архитектура программы:

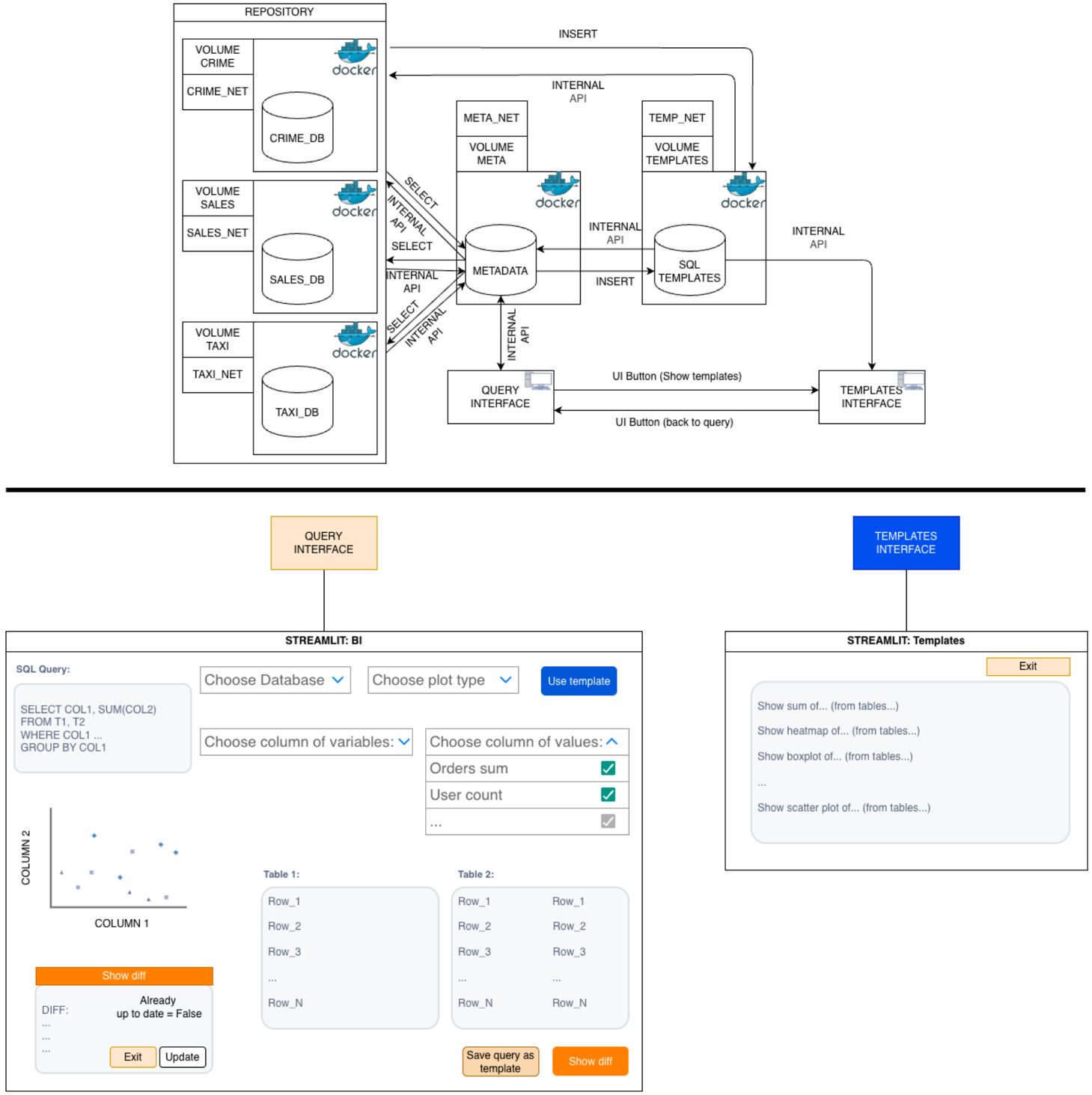



Рис. 6: Архитектура программы

Цвета кнопок и названий элементов интерфейса связаны переходами между элементами интерфейса (синяя кнопка use template -> интерфейс templates).

В самом интерфейсе есть поля выбора базы данных источника, далее возможность выбора типа графика (столбчетая диаграмма, обычный график, график разброса, heatmap). Ниже находятся кнопки просмотра отличий данных из баз данных источников. Слева сверху отображается сам SQL запрос, генерируемый путем заполнения форм. Запрос можно сохранить текстом в базу данных шаблонов, после чего использовать шаблоны повторно.

На рисунке 7, показана модульная схема бизнес процессов, автоматизированных с помощью разрабатываемого ПО:

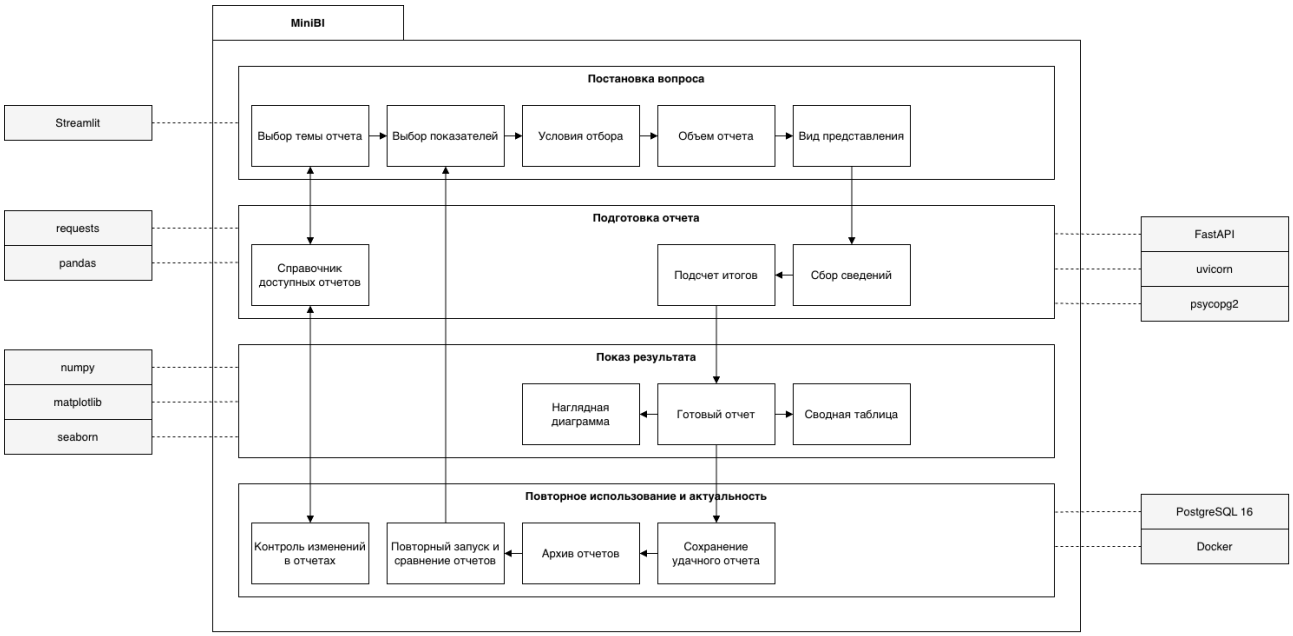



Рис. 7: Модульная архитектура бизнес процессов

# 7 Разработка системы

Слой доступа к данным вынесен в отдельный модуль и построен так, что Python отвечает только за подстановку имен и вывод, а вся работа с данными остается на стороне СУБД. Для связи с postgres используется драйвер psycopg третьей версии, так как с его помощью в дальнейшем данные можно будет легко переводить в массивы для построения графиков. Модуль разбит на три части: соединение и чтение окружения, текстовые шаблоны запросов, перевод результата в массивы.

## 7.1 Соединение

Каждая база живет в своем контейнере, наружу проброшен только порт postgres, поэтому с точки зрения слоя доступа подключение ничем не отличается от обычного локального: адрес, порт, имя базы, пользователь и пароль. Соединение открывается на время одного запроса и закрывается контекстным менеджером.

```
def catalog(source_id):
    with psycopg.connect(row_factory=dict_row, **meta()) as connection:
        with connection.cursor() as cursor:
            cursor.execute(template.CATALOG_SQL, (source_id,))
            return cursor.fetchall()
```

Листинг 1: Чтение каталога метаданных

Ошибки драйвера перехватываются и возвращаются текстом, то есть недоступная база или неверное имя колонки приводят к выводу сообщения, после чего запрос о вводе имени повторяется.

## 7.2 Окружение

Все параметры подключения собраны в одном файле credentials.env.

Переменные сгруппированы по префиксу источника, для каждой базы задаются хост, порт, имя, пользователь, пароль и образ. Слой доступа разбирает файл сам и собирает словарь источников по суффиксу имени переменной: встретив ключ, оканчивающийся на DB_NAME, он достает по тому же префиксу остальные параметры. Благодаря этому добавление еще одной базы не требует правок кода, достаточно дописать блок переменных.

## 7.3 Передача данных

Данные и метаданные приходят из разных баз. Сначала выполняется запрос к базе метаданных, откуда берется описание выбранного источника, затем по этому описанию собирается запрос и отправляется уже в саму базу-источник. Такое разделение и позволяет системе оставаться независимой от источников.

Передаются при этом текст запроса в одну сторону и результат в другую. Рядом с каждой базой поднят свой REST, который принимает запрос методом POST, выполняет его и возвращает ответ в формате JSON из списка имен колонок и списка строк. Отдельным методом агент отдает описание своих таблиц и колонок, откуда база метаданных и наполняет каталог.

### 7.4 Шаблонный запрос

Запрос к источнику описан одной строкой-шаблоном с именованными местами подстановки:

```
SELECT {group_cols}, {agg_func}({agg_arg}) AS value
FROM {table}
GROUP BY {group_cols}
ORDER BY value DESC
LIMIT 20
```

Листинг 2: Шаблон запроса к источнику

Подстановка текстовая так как таблицу или колонку так передать нельзя. Чтобы подстановка не давала ошибок каждое имя перед этим проверяется по каталогу метаданных, и в шаблон попадает только то, что там найдено.

Полное имя собирается из полей schema_name и table_name строки каталога:

```
SELECT schema_name, table_name, column_name, data_type,
       pk_key, fk_key, ref_table_name, ref_column_name
FROM meta.catalog
WHERE source_id = %s
ORDER BY table_name, ordinal_position
```

Листинг 3: Запрос к каталогу метаданных

Порядок строк каталога совпадает с порядком колонок в исходной таблице, что позволяет показывать атрибуты так, как они объявлены в источнике. Поля ключей и ссылок в текущем варианте шаблона не участвуют в сборке, но читаются вместе с остальными, так как описывают связи источника и нужны при расширении запроса на несколько таблиц.

#### 7.4.1 Агрегатные функции

Допустимых функций три: SUM, AVG и COUNT, и выбор ограничен списком, то есть в шаблон не может попасть произвольное слово.

Аргументом служит колонка значений, выбранная пользователем и проверенная по каталогу. Если колонка не задана, агрегат вырождается в COUNT по всем строкам, и вместо имени подставляется звездочка. То есть когда суммировать нечего, считается количество записей.

Тип колонки при этом берется из поля data_type каталога, то есть система знает, что числовое, а что нет, не обращаясь к самому источнику.

#### 7.4.2 Условие

Условие можно наложить только на выбранные колонки категорий и значений. Каждое условие хранится тройкой из имени колонки, оператора и значения, а в запрос они попадают через AND между шаблоном FROM и GROUP BY, то есть отбор строк выполняется до группировки.

Какие операторы доступны, решает поле data_type каталога. Для числовых колонок оператор выбирается из фиксированного списка $=, <, >, \leq, \geq$ и значение подставляется как есть. Для остальных оператор не выбирается, условие собирается как LIKE по вхождению подстроки, а одинарные кавычки в значении экранируются удвоением:

```
for col, op, value in filters:
    if op == "LIKE":
        escaped = str(value).replace("'", "''")
        conds.append(f"{col} LIKE '%{escaped}%'")
    elif op in COMPARISON_OPS:
        conds.append(f"{col} {op} {value}")
```

Листинг 4: Сборка условий

Если ни одно условие не задано, WHERE в запрос не добавляется вовсе. Тот же набор условий используется и для запроса дополнительных колонок, поэтому обе таблицы результата показывают одни и те же строки.

### 7.4.3 Ограничение по строкам

Ограничение задается текстовым полем и дописывается последней строкой запроса, после GROUP BY, так что его выполняет сам источник и через REST передается уже урезанный результат.

Пустое поле или не число заменяются значением по умолчанию 500, чтобы случайно не выгрузить всю таблицу. Ноль или отрицательное число означают отсутствие ограничения, и тогда LIMIT не добавляется. Для запроса дополнительных колонок, где группировки нет и строк может быть много, при отсутствии ограничения подставляется LIMIT 200.

При смене базы, таблицы или колонок поле сбрасывается, так как старое ограничение к новому запросу обычно не относится.

### 7.4.4 Группировка

Группировка определяется набором выбранных атрибутов. Основная категориальная колонка плюс дополнительные, если они заданы. Этот набор подставляется в шаблон дважды, в список выборки и в GROUP BY, из одного и того же места, поэтому запрос не может оказаться несогласованным, когда колонка выбирается, но не группируется.

Результат агрегата всегда получает фиксированное имя value и по нему же сортируется по убыванию. Фиксированное имя избавляет последующую обработку от разбора заголовков. То есть последняя колонка результата это значение, а все предыдущие это подписи.

## 7.5 Сохранение шаблонов запросов

Шаблоны хранятся в отдельной базе tpl, к которой, как и к источникам, обращаются через свой REST. Он дает четыре метода: получить список шаблонов, создать шаблон, отметить использование и удалить.

При сохранении отправляется готовый текст запроса вместе с параметрами формы: источник, схема, таблица, вид графика, колонки, агрегат и лимит. Условия сохраняются дважды: сами уже есть в тексте запроса, а в поле filter_summary кладется их читаемая сводка для списка шаблонов. Если имя не задано, его выдает триггер при вставке:

```
IF NEW.template_name IS NULL OR btrim(NEW.template_name) = '' THEN
    NEW.template_name := 'template_' || nextval('tpl.template_name_seq');
END IF;
```

Листинг 5: Имя шаблона по умолчанию

Проверки на уровне таблицы не дают сохранить неизвестный вид графика или агрегат, неположительный лимит и запрос, начинающийся не с SELECT. При применении шаблона параметры переносятся обратно в форму, а в базе увеличивается счетчик used_count и обновляется last_used_at, по которому потом можно удалять неиспользуемые шаблоны.

## 7.6 Перевод в NumPy массив

Курсор возвращает результат в виде списка кортежей и отдельного списка имен колонок для вывода таблиц. А для построения графика данные нужно разложить на две однотипные последовательности. Последняя колонка это агрегат, остальные это подписи. Подписи склеиваются в строку, значения приводятся к числу с плавающей точкой, после чего обе последовательности оборачиваются в массивы NumPy:

```
def to_arrays(rows):
    labels = np.array([" ".join(str(cell) for cell in row[:-1]) for row in
       rows])
    values = np.array([float(row[-1]) for row in rows])
    return labels, values
```

Листинг 6: Перевод результата в массивы

Перевод к float так как в postgres numeric драйвер отдает как Decimal, с которым графические библиотеки работать не будут. Массивы NumPy выбраны потому, что именно их принимают функции построения графиков, и потому что индексация массива позволяет задать позиции столбцов и точек по оси абсцисс.

## 7.7 Построение графиков

Визуализация построена на matplotlib. Поддерживаются два вида графиков: столбчатый, показывающий соотношение агрегата между категориями, и точечный, удобный когда хочется посмотреть разброс значений.

По оси абсцисс откладываются порядковые номера, полученные через arange, подписи же навешиваются на деления отдельно и поворачиваются, иначе длинные категориальные значения накладываются друг на друга.

## 7.8 Запуск

Каждый контейнер поднимается своим compose-файлом, а параметры подключения берутся из общего файла окружения, который передается ключом env-file.

```
docker-compose --env-file credentials.env -f databases/new/
    extra_for_students/docker-compose.yml up -d
docker-compose --env-file credentials.env -f databases/new/mountain_sport/
    docker-compose.yml up -d
docker-compose --env-file credentials.env -f databases/new/
    sport_and_cheerleaders/docker-compose.yml up -d
docker-compose --env-file credentials.env -f information-schema/docker-
    compose.yml up -d
docker-compose --env-file credentials.env -f templates-db/docker-compose.yml
     up -d
```

Листинг 7: Запуск контейнеров

Порядок важен - база метаданных подключается к сетям источников как к внешним, поэтому сети должны быть созданы раньше. Внутри стека порядок обеспечивает сам compose связи стартуют только после того, как проверка готовности баз прошла успешно. При остановке порядок обратный, иначе Docker не даст удалить сети источников.

Приложение запускается отдельно, вне Docker, из окружения conda с установленными psycopg, numpy и matplotlib:

```
python temp/app.py
```

Листинг 8: Запуск приложения

Ниже показан пример сокращенного вывода (см. листинг 9) и ее графика (см. рис. 8):

```
Which db ?
    crime
Which table ?
    incidents
Columns:
    incident_id ... ward community_area year
    Variable ? year
    Values ? ward
    Agg func ? count
    Plot type ? bar

SELECT crime.incidents.year, COUNT(crime.incidents.ward) AS value
FROM crime.incidents
GROUP BY crime.incidents.year
ORDER BY value DESC
LIMIT 20

year  value
2021  806
...
2010  438
```

Листинг 9: Пример работы

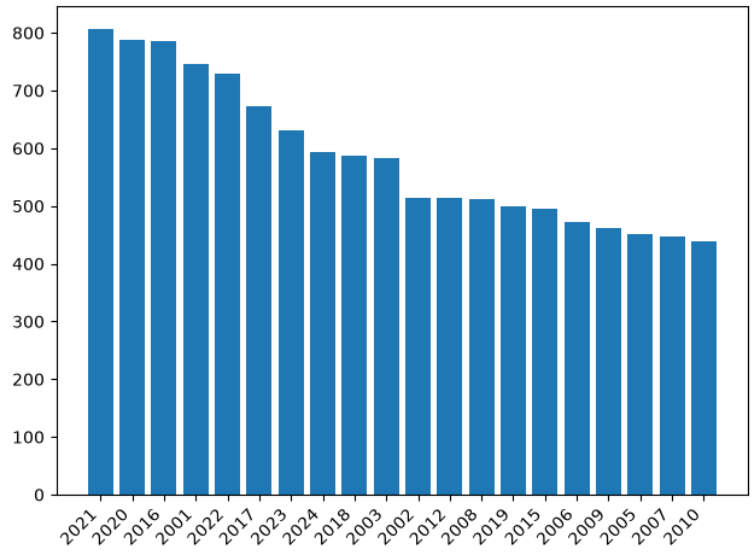



Рис. 8: Пример вывода графика

# 8 Интерфейс

Для создания минимальной версии BI системы, было необходимо разработать пользовательский интерфейс включающий в себя: выбор конкретной базы данных источников (или все базы), выбор таблиц из источников, выбор аггрегатной функции, выбор типа графика, ввод условий, ввод количества вывода строк, и вывод основных и дополнительных таблиц.

Интерфейс собран на streamlit и занимает одну страницу, разделенную на две колонки: справа форма с элементами управления, слева текст собранного запроса и график по нему. Под ними выводятся таблицы с результатом.

## 8.1 Формы

Основные параметры задаются выпадающими списками, так как все допустимые значения известны заранее из каталога метаданных: выбор базы данных, выбор таблицы, выбор вида графика, выбор колонки категорий, выбор колонки значений и выбор агрегатной функции. Список колонок показывает рядом с именем короткое обозначение типа, что бы было понятно, что можно суммировать, а что нет.

Ограничение на количество строк задается текстовым полем. Условие отбора вынесено во всплывающую панель, где для каждой колонки стоит выпадающий список с оператором сравнения и текстовое поле для значения; для строковых колонок оператор не выбирается.

Ниже показаны интерфейсы форм где:

- Красный цвет - кнопка;
- Желтный цвет - выпадающий список;
- Зеленый цвет - статический вывод;
- Синий цвет - поле ввода.

## 8.2 Выбор нескольких колонок

Отдельная всплывающая панель содержит перечень остальных колонок таблицы, каждая с флажком и подписью типа. Она нужна, что бы кроме агрегированного результата посмотреть исходные значения: отмеченные колонки выводятся второй таблицей рядом с основной. На рисунке 9 показан интерфейс выбора дополнительных колонок.

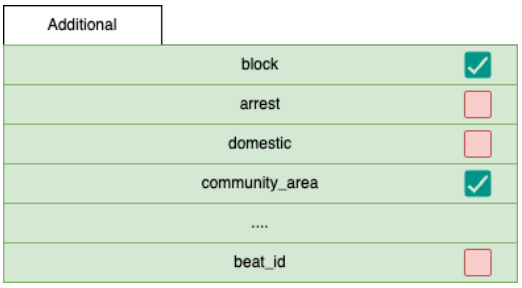



Рис. 9: Интерфейс отличий

## 8.3 Вывод таблиц

Результат показывается табличным блоком с прокруткой и постраничным просмотром: номер страницы задается числовым полем, под таблицей выводится подпись с диапазоном показанных строк и общим их числом. Основная и дополнительная таблицы располагаются рядом в двух колонках.

## 8.4 Вывод графиков

График строится в левой колонке сразу под текстом запроса, который выводится отдельным блоком кода, то есть видно и результат, и то, каким запросом он получен. Вид графика выбирается в форме: столбчатый, круговой, точечный и тепловая карта. Если для выбранных колонок график построить нельзя, вместо него выводится сообщение.

## 8.5 Вывод SQL запроса

Вывод финального SQL запроса строится в отдельном блоке слева сверху, обычным текстом.

## 8.6 Форма условий

Форма условий включает в себя отображение основных колонок, их типы и выпадающий список операторов: $\leq, \geq, >, <, =$. Ниже показан интерфейс формы условий 10.

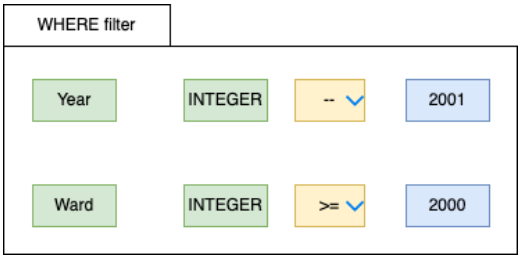



Рис. 10: Интерфейс отличий

## 8.7 Вывод отличий

Сравнение сохраненного каталога с реальной схемой источника вызывается кнопкой и открывается отдельным модальным окном. Отличия сгруппированы по типу: добавленные, измененные и удаленные колонки, каждая группа выводится своей таблицей. Там же находится кнопка обновления каталога, неактивная, когда расхождений нет. Ниже на рисунке 11 показан интерфейс.

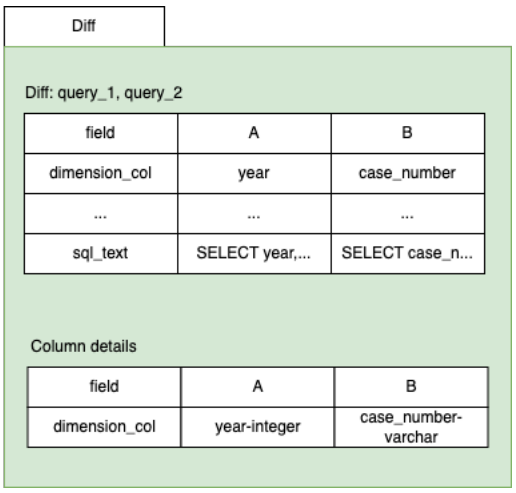



Рис. 11: Интерфейс отличий

## 8.8 Шаблон интерфейса

Ниже на рисунке 12 продемонстрирован шаблон интерфейса собранный из форм и выводом графика. В последствие будет реализована рабочая версия интерфейса.

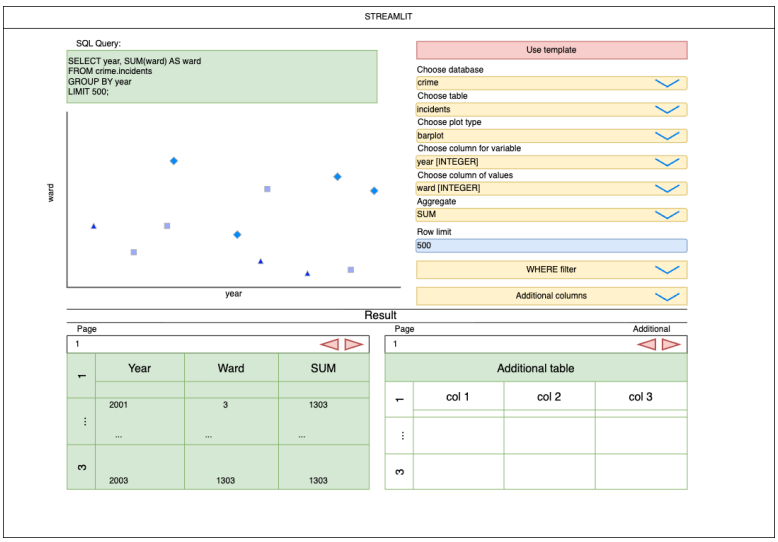



Рис. 12: Шаблон интерфейса

# 9 Разработка интерфейса

Интерфейс реализован на Streamlit и разделен на две части: справа находятся элементы управления и слева выводится SQL запрос и график, а снизу таблицы результата. Каждый элемент управления хранит свое значение в состоянии сессии (st.session_state), и при любом изменении страница перерисовывается. Выбранные значения собираются в один словарь параметров (источник, таблица, вид графика, колонки, агрегатная функция, условия, лимит), который передается в функцию построения SQL запроса, после чего запрос отправляется RESTy источника.

## 9.1 Поля ввода

Текстовое поле используется там, где значение заранее неизвестно и его нельзя выбрать из списка. Основное поле это Row limit - ограничение на количество строк, которое подставляется в LIMIT запроса. Если поле пустое или введено не число, используется значение по умолчанию 500, поэтому неправильный ввод не ломает запрос. При смене таблицы или колонок поле сбрасывается, что бы лимит от прошлого запроса не переносился на новый. Так же поля ввода есть в форме условий (значение для сравнения) и под таблицами результата (номер страницы). Поле создается методом st.text_input, а номер страницы под таблицами - методом st.number_input. Ниже на рисунке 13 показано само поле ввода.

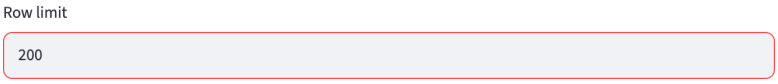



Рис. 13: Итоговый интерфейс поля ввода

## 9.2 Выпадающие списки

Выпадающие списки используются для всех параметров, допустимые значения которых заранее известны из meta.catalog: выбор базы данных, таблицы, вида графика (barplot, pieplot, scatter, heatmap), колонки переменной, колонки значений и агрегатной функции (SUM, AVG, COUNT). Так пользователь не может выбрать несуществующую таблицу или колонку. Рядом с именем колонки в квадратных скобках выводится короткое обозначение типа данных, которое берется из data_type каталога, что бы было понятно какую колонку можно суммировать, считать среднее и т.д. После выбора базы список таблиц содержит только ее таблицы, а колонка значений не может совпадать с колонкой переменной. Для heatmap появляется дополнительный список второй размерности, а для scatter список агрегатной функции скрывается, так как точки выводятся без группировки. Все выпадающие списки создаются методом st.selectbox. Итоговый интерфейс выпадающего списка показан на рисунке 14.

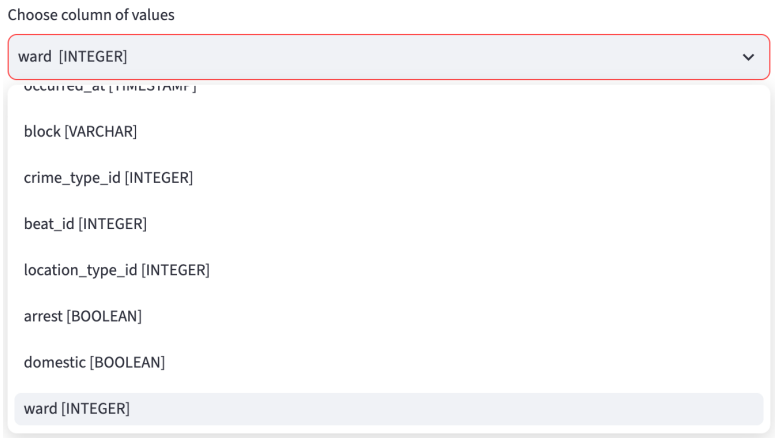



Рис. 14: Итоговый интерфейс выпадающего списка

## 9.3 Кнопки

Кнопки отвечают за действия которые открывают отдельные окна (см. рис. 15):

- Use template - открывает окно со списком сохраненных шаблонов из которого шаблон можно применить, удалить или сравнить с другим;
- Save query as template - сохраняет текущий запрос и параметры формы в базу шаблонов;
- Show diff - сравнивает сохраненный meta.catalog с реальной схемой источника.

Кнопки WHERE filter и Additional columns открывают всплывающие панели, они описаны ниже. Кнопки создаются методом st.button, а открываемые ими окна - декоратором @st.dialog.




Рис. 15: Итоговый интерфейс кнопок

## 9.4 Выбор нескольких атрибутов

Панель Additional columns содержит все остальные колонки выбранной таблицы (кроме колонки переменной и колонки значений), каждая с флажком и типом. Она нужна что бы кроме агрегированного результата посмотреть исходные строки. По отмеченным колонкам строится отдельный запрос без группировки с теми же условиями и лимитом, и его результат выводится второй таблицей Details рядом с основной. Панель создается методом st.popover, флажки - методом st.checkbox, а таблица Details выводится методом st.dataframe. Ниже на рисунке 16 показан выбор нескольких атрибутов.

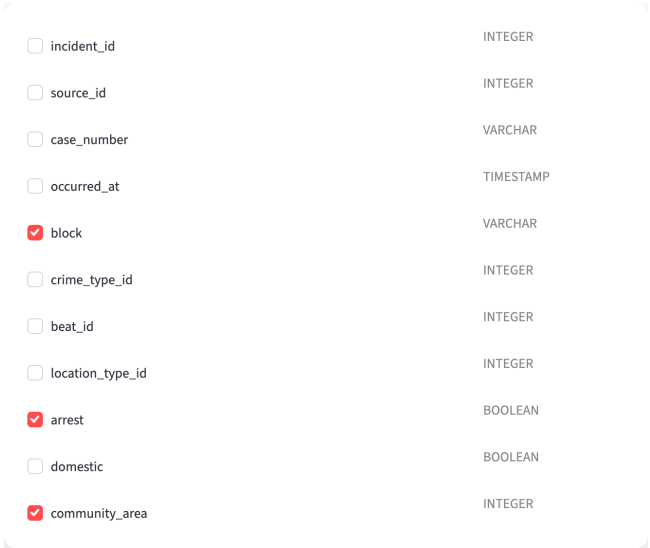



Рис. 16: Итоговый интерфейс выбора нескольких атрибутов

## 9.5 Форма выбора шаблона

Форма открывается кнопкой Use template в окне и выводит все шаблоны из базы шаблонов. Каждая строка содержит кнопку с описанием шаблона (имя, источник и таблица, вид графика и колонки), флажок compare и кнопку Delete. При нажатии на шаблон его параметры записываются в состояние сессии, то есть все выпадающие списки и поле лимита заполняются значениями

из шаблона, и запрос строится заново, а в базе шаблонов увеличивается счетчик использования. Окно создается декоратором @st.dialog, шаблоны и кнопка Delete - методом st.button, флажок compare - методом st.checkbox. Итоговый интерфейс формы показан на рисунке 17.

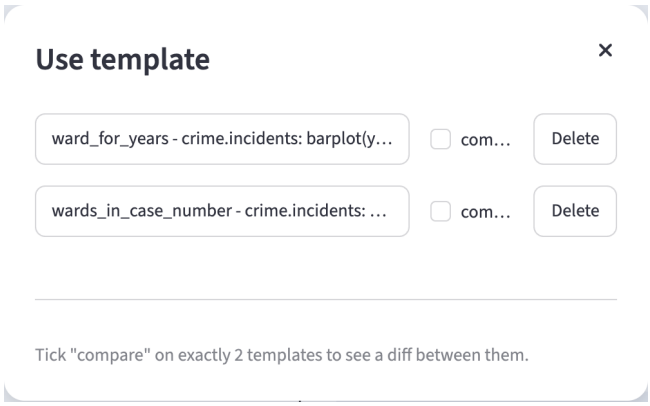



Рис. 17: Итоговый интерфейс формы выбора шаблона

## 9.6 Форма для условий

Форма WHERE filter открывается всплывающей панелью и содержит строку для колонки переменной и колонки значений. В строке выводится имя колонки, ее тип, выпадающий список оператора и поле для значения. Для числовых колонок доступны операторы $=, <, >, \leq, \geq$, значение - означает, что условие не задано. Для строковых колонок введенный текст ищется через LIKE. Условие добавляется в запрос только если выбран оператор и введено значение, несколько условий объединяются через AND. На рисунке 18 задано условие year $\geq$ 2010, которое в запросе превращается в WHERE year >= 2010 (см. рис. 21). Панель создается методом st.popover, оператор выбирается через st.selectbox, значение вводится через st.text_input.

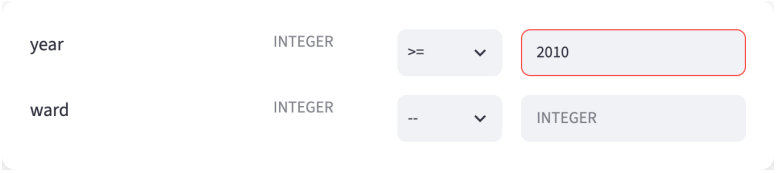



Рис. 18: Итоговый интерфейс формы условий

## 9.7 Форма для сравнений

В интерфейсе есть два вида сравнения. Первое - сравнение двух шаблонов: в форме выбора шаблона нужно отметить флажок compare у двух шаблонов, и ниже появится таблица отличающихся полей (колонки, условия, лимит, текст запроса) и таблица Column details, где для колонок шаблонов из meta.catalog подставлен их тип (см. рис. 19). Таблицы отличий выводятся методом st.dataframe.

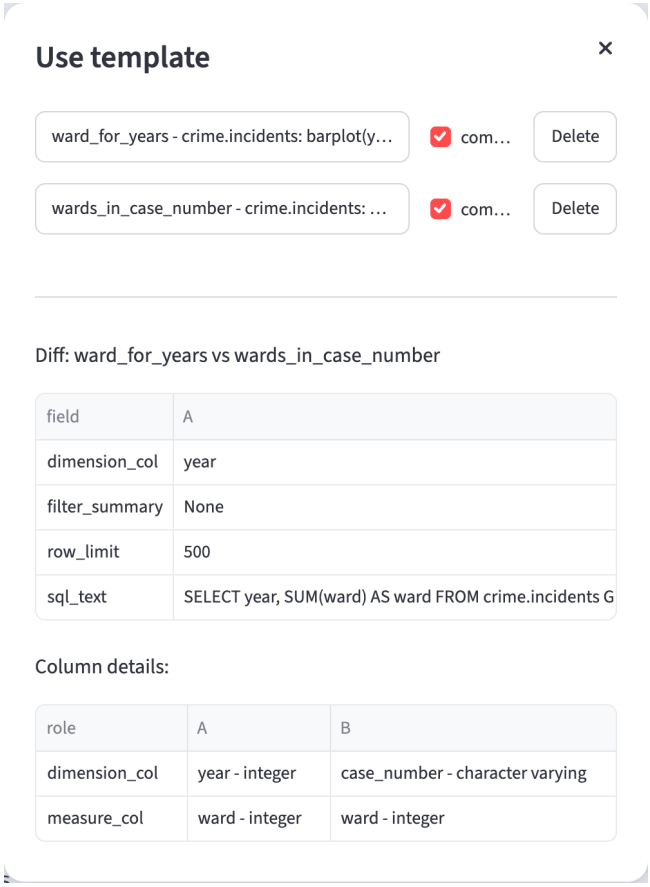



Рис. 19: Итоговый интерфейс сравнения шаблонов

Второе - сравнение каталога с источником (Show diff). Окно показывает количество добавленных, измененных и удаленных колонок и таблицу для каждой группы. Кнопка Update синхронизирует meta.catalog с источником и неактивна, если отличий нет, как на рисунке 20. Окно создается декоратором @st.dialog, количество отличий выводится через st.write, а таблицы групп - через st.dataframe.

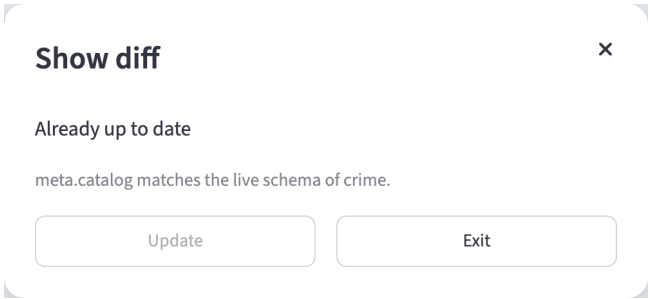



Рис. 20: Итоговый интерфейс сравнения каталога с источником

## 9.8 Результат

На рисунке 21 показан итоговый интерфейс. По умолчанию данные не заполнены. Можно выбрать: базу данных, таблицу, ее атрибуты, тип графика. Далее ввести условия и лимит выводимых строк, после чего нажать кнопку «Generate report».

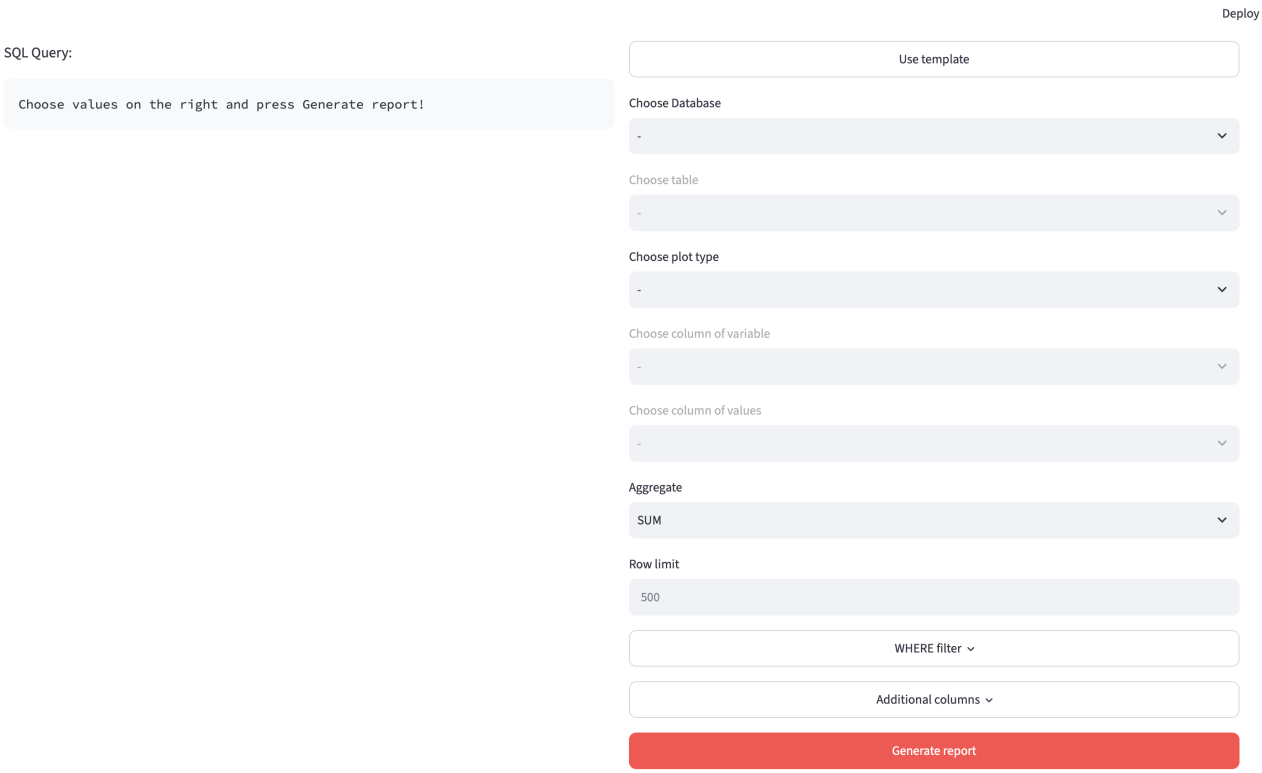



Рис. 21: Итоговый интерфейс

# 10 Результаты работы

Ниже в главах 10.1 - 10.11, представлены 11 сценариев использования разработанной минимальной BI-системы с интерфейсом и итоговым запросом.

## 10.1 Сценарий 1

Вывод первых 500 результатов горнолыжников из таблицы result базы mountain_sport, где attempt1_result $> 0$, в виде диаграммы рассеивания времени первой попытки (attempt1_result) против второй (attempt2_result). Итоговый SQL запрос приведен в листинге 10, результат на рисунке 22.

```sql
SELECT attempt1_result, attempt2_result
FROM mountain_sport.result
WHERE attempt1_result > 0
LIMIT 500
```

Листинг 10: Запрос сценария 1

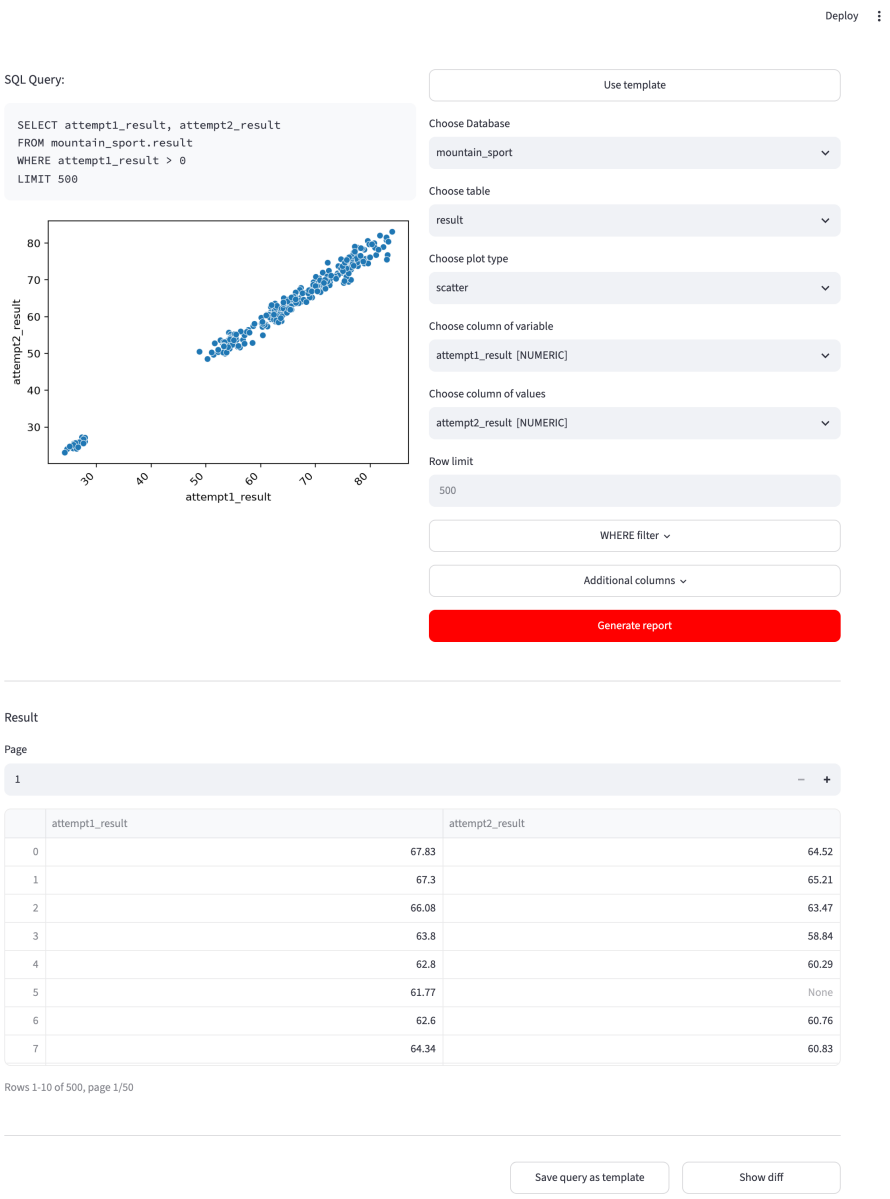



Рис. 22: Сценарий 1

## 10.2 Сценарий 2

Вывод столбчатой диаграммы суммарной стоимости занятий по предметам subject_id из таблицы lesson базы extra_for_students, где стоимость cost $\geq$ 1000 рублей. Итоговый SQL запрос приведен в листинге 11, результат на рисунке 23.

```sql
SELECT subject_id, SUM(cost) AS cost
FROM extra_for_students.lesson
WHERE cost >= 1000
GROUP BY subject_id
LIMIT 500
```

Листинг 11: Запрос сценария 2

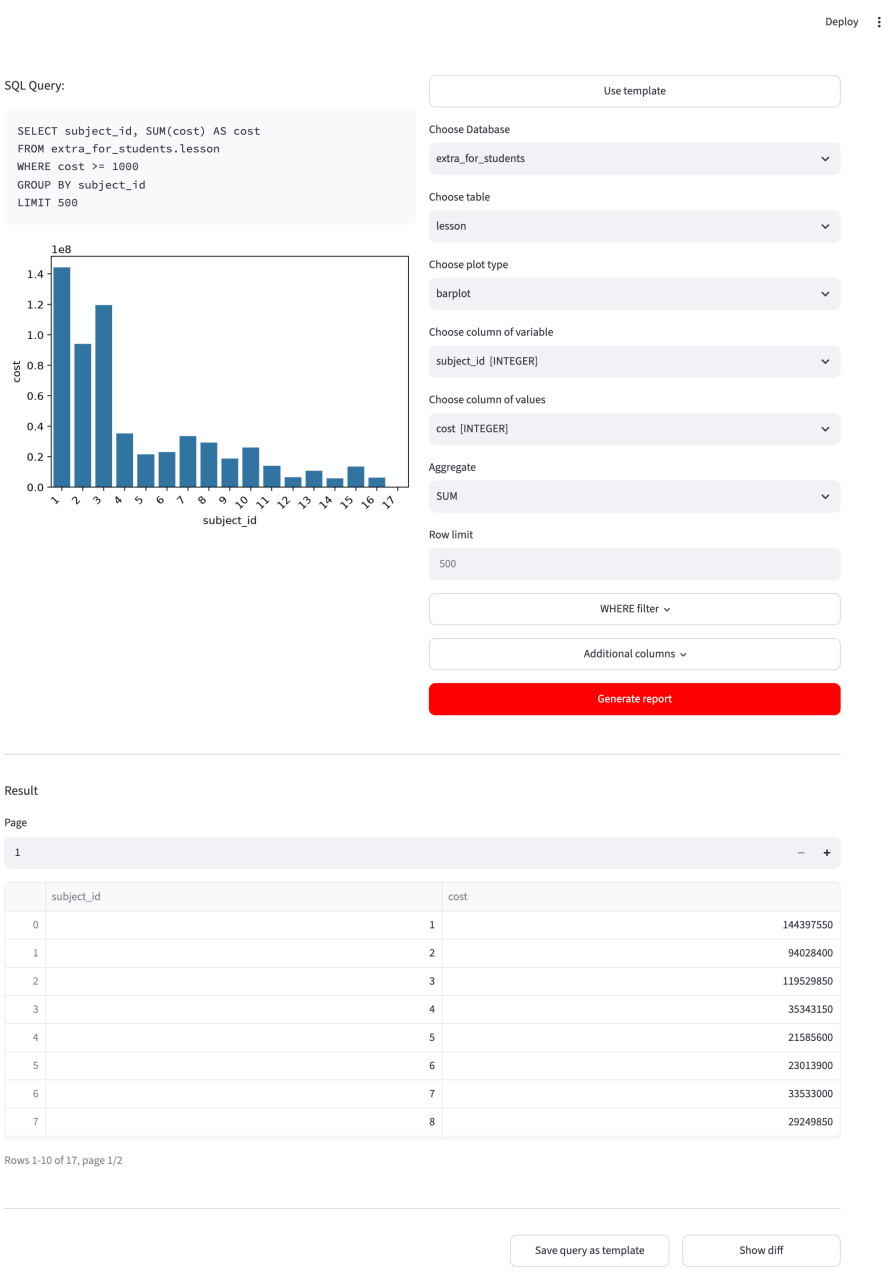



Рис. 23: Сценарий 2

## 10.3 Сценарий 3

Вывод круговой диаграммы доли выступлений по номинациям по perfomance_type из таблицы perfomance базы sport_and_cheerleaders. Итоговый SQL запрос приведен в листинге 12, результат на рисунке 24.

```
SELECT perfomance_type, COUNT(perfomance_id) AS perfomance_id
FROM sport_and_cheerleaders.perfomance
GROUP BY perfomance_type
LIMIT 500
```

Листинг 12: Запрос сценария 3

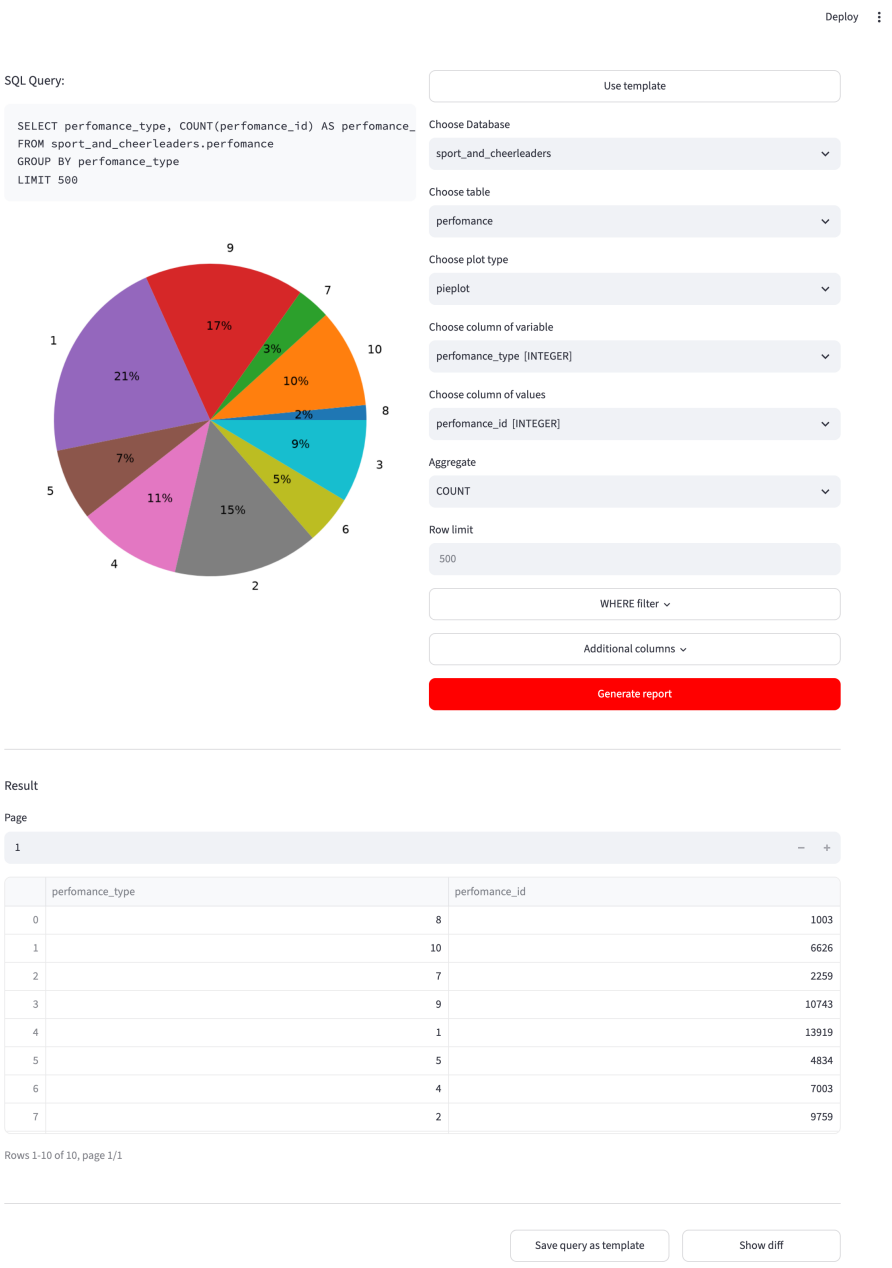


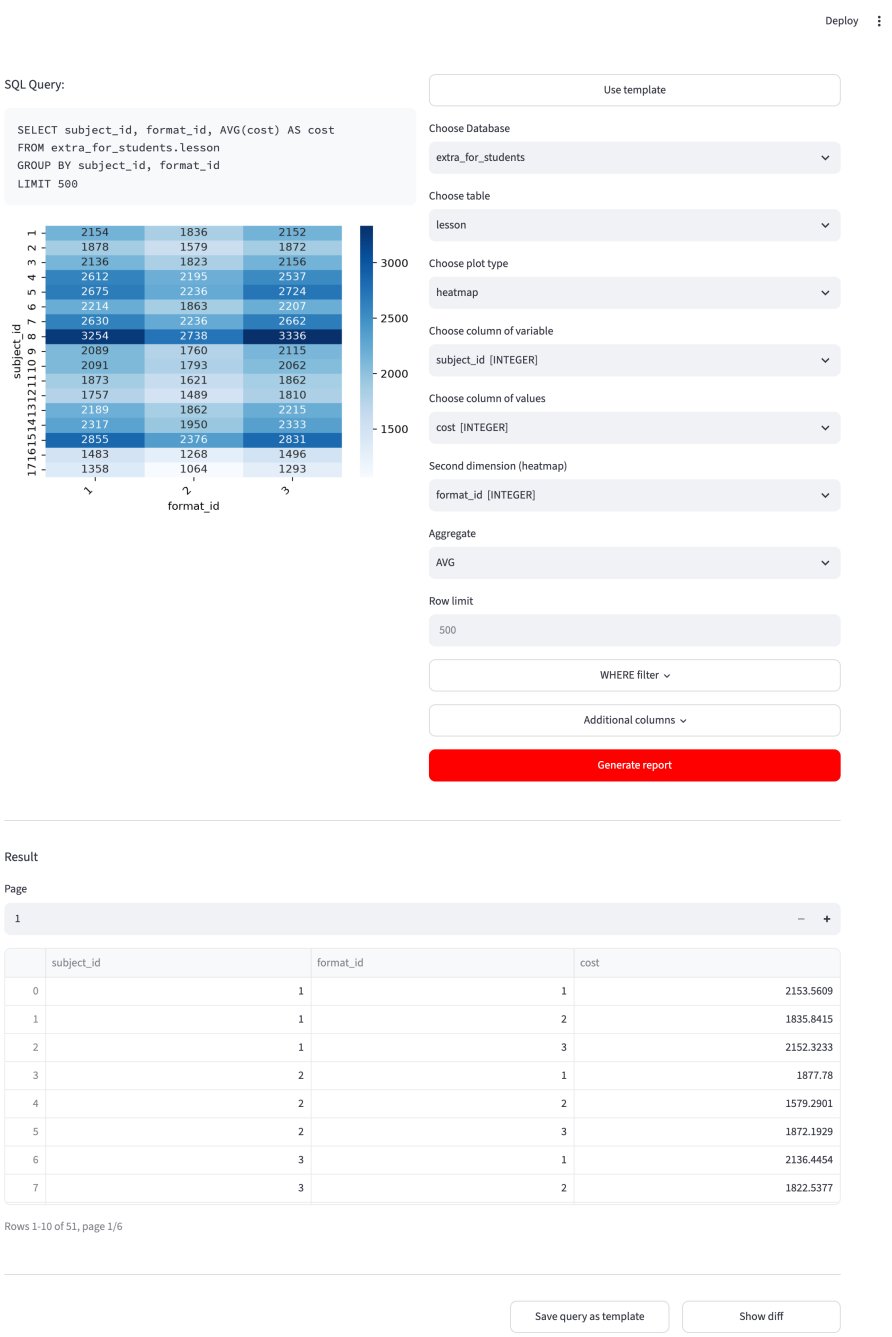

Рис. 24: Сценарий 3

## 10.4 Сценарий 4

Вывод тепловой карты средней стоимости занятия по предметам subject_id и форматам format_id из таблицы lesson базы extra_for_students. Итоговый SQL запрос приведен в листинге 13, результат на рисунке 25.

```sql
SELECT subject_id, format_id, AVG(cost) AS cost
FROM extra_for_students.lesson
GROUP BY subject_id, format_id
LIMIT 500
```

Листинг 13: Запрос сценария 4




| | subject_id | format_id | cost |
|---|---|---|---|
| 0 | 1 | 1 | 2153.5609 |
| 1 | 1 | 2 | 1835.8415 |
| 2 | 1 | 3 | 2152.3233 |
| 3 | 2 | 1 | 1877.78 |
| 4 | 2 | 2 | 1579.2901 |
| 5 | 2 | 3 | 1872.1929 |
| 6 | 3 | 1 | 2136.4454 |
| 7 | 3 | 2 | 1822.5377 |

Рис. 25: Сценарий 4

### 10.5 Сценарий 5

Применение сохраненного шаблона lesson_cost_by_subject (сумма стоимости занятий cost по предметам subject_id из таблицы lesson базы extra_for_students) через кнопку Use template. В окне выводится список сохраненных шаблонов (см. рис. 26), после нажатия на шаблон его параметры переносятся в форму и отчет строится сразу (см. рис. 27). SQL запрос шаблона приведен в листинге 14.

```
SELECT subject_id, SUM(cost) AS cost
FROM extra_for_students.lesson
GROUP BY subject_id
LIMIT 500
```

Листинг 14: Запрос сценария 5

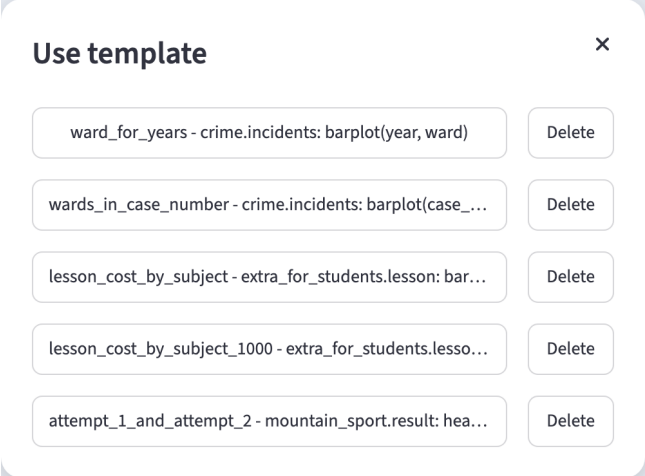



Рис. 26: Сценарий 5: список сохраненных шаблонов

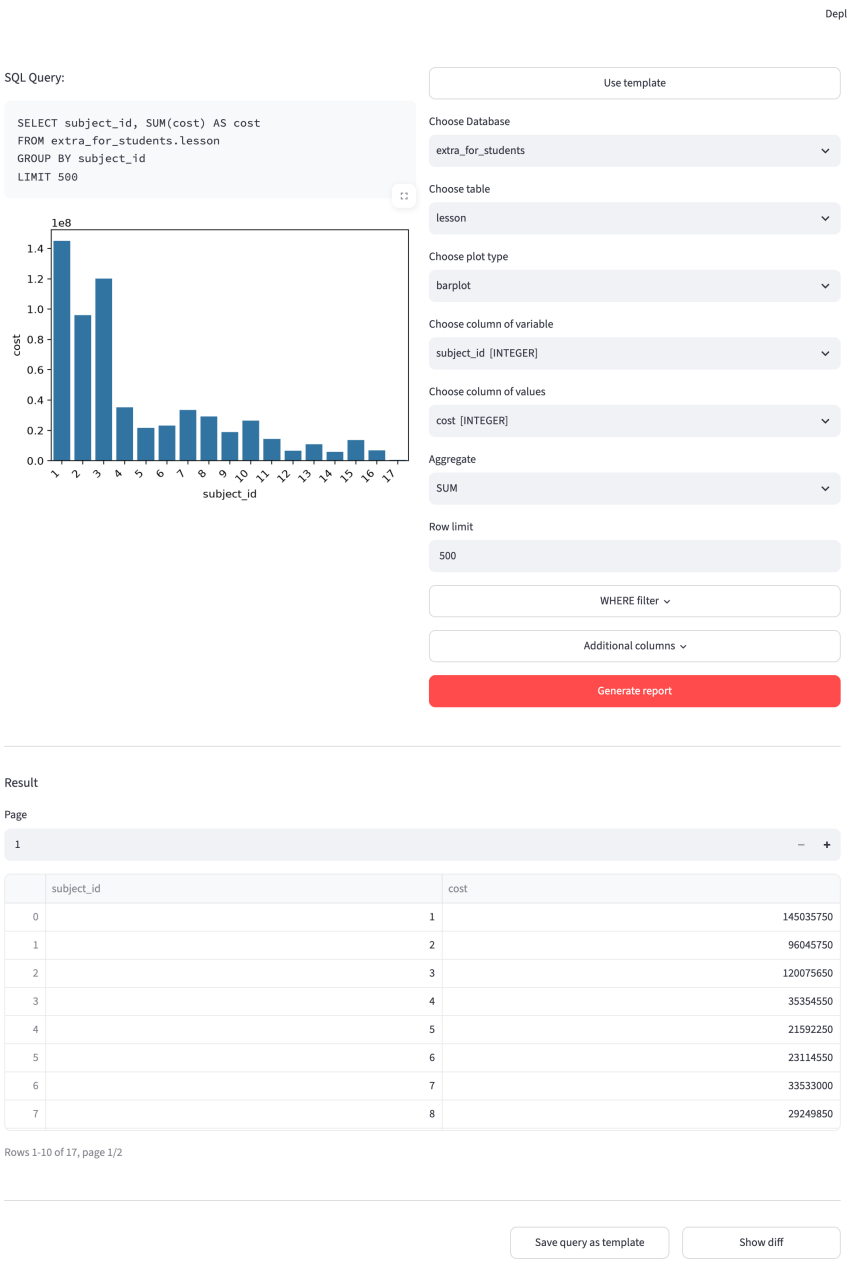



Рис. 27: Сценарий 5: отчет по примененному шаблону

## 10.6 Сценарий 6

Добавление в источнике mountain_sport новой колонки в таблицу athlete, ее обнаружение кнопкой Show diff как добавленной и синхронизация meta.catalog кнопкой Update. Изменение источника приведено в листинге 15, найденное отличие и состояние после обновления на рисунках 28 и 29.

```
1 ALTER TABLE mountain_sport.athlete ADD COLUMN height SMALLINT;
```

Листинг 15: Запрос сценария 6

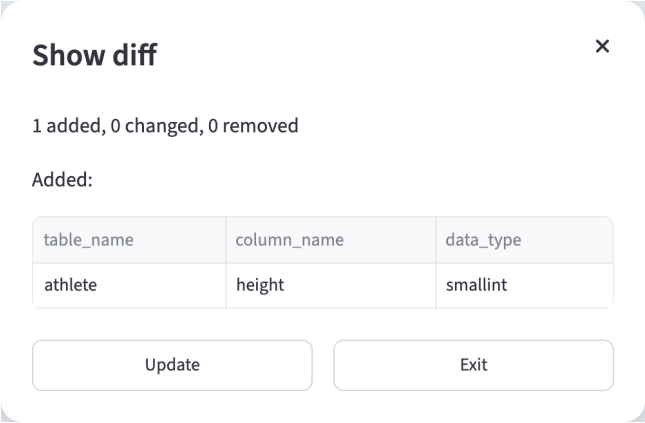



Рис. 28: Сценарий 6: найденное отличие источника

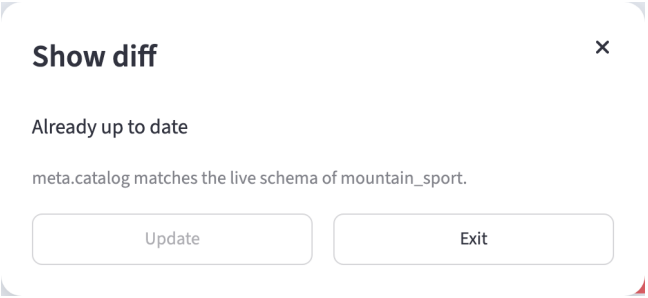



Рис. 29: Сценарий 6: каталог после обновления

### 10.7 Сценарий 7

Вывод количества спортсменов по клубам club_id из таблицы sportsman базы sport_and_cheerleaders, где club_id $\leq 50$, с дополнительными колонками sportsman_lastname и sportsman_birthdate в таблице Details. Итоговый SQL запрос приведен в листинге 16, результат на рисунке 30.

```
SELECT club_id, COUNT(sportsman_id) AS sportsman_id
FROM sport_and_cheerleaders.sportsman
WHERE club_id <= 50
GROUP BY club_id
LIMIT 500
```

Листинг 16: Запрос сценария 7

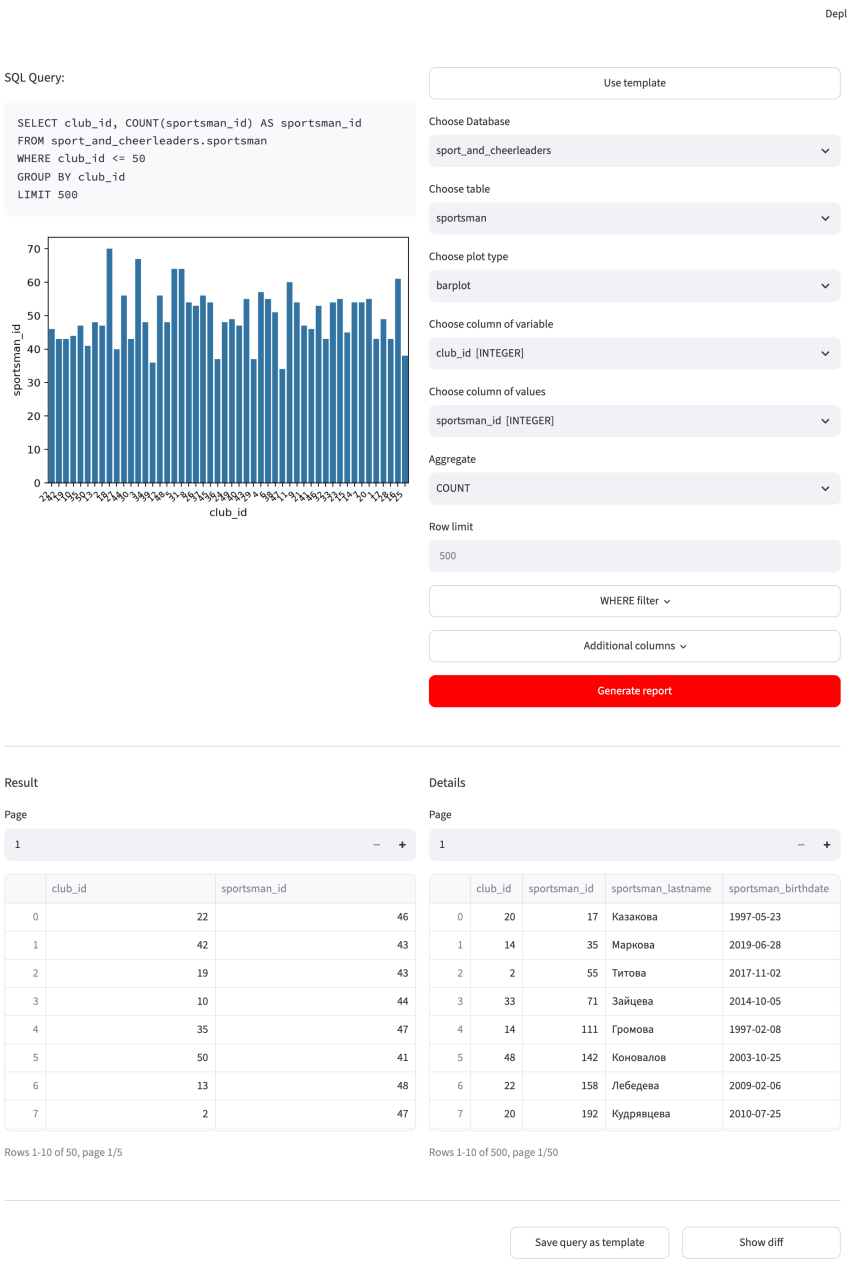



Рис. 30: Сценарий 7

## 10.8 Сценарий 8

Вывод среднего опыта репетиторов по городам city_id из таблицы tutors базы extra_for_students, где опыт experience $\geq 5$ лет. Итоговый SQL запрос приведен в листинге 17, результат на рисунке 31.

```sql
SELECT city_id, AVG(experience) AS experience
FROM extra_for_students.tutors
WHERE experience >= 5
GROUP BY city_id
LIMIT 500
```

Листинг 17: Запрос сценария 8

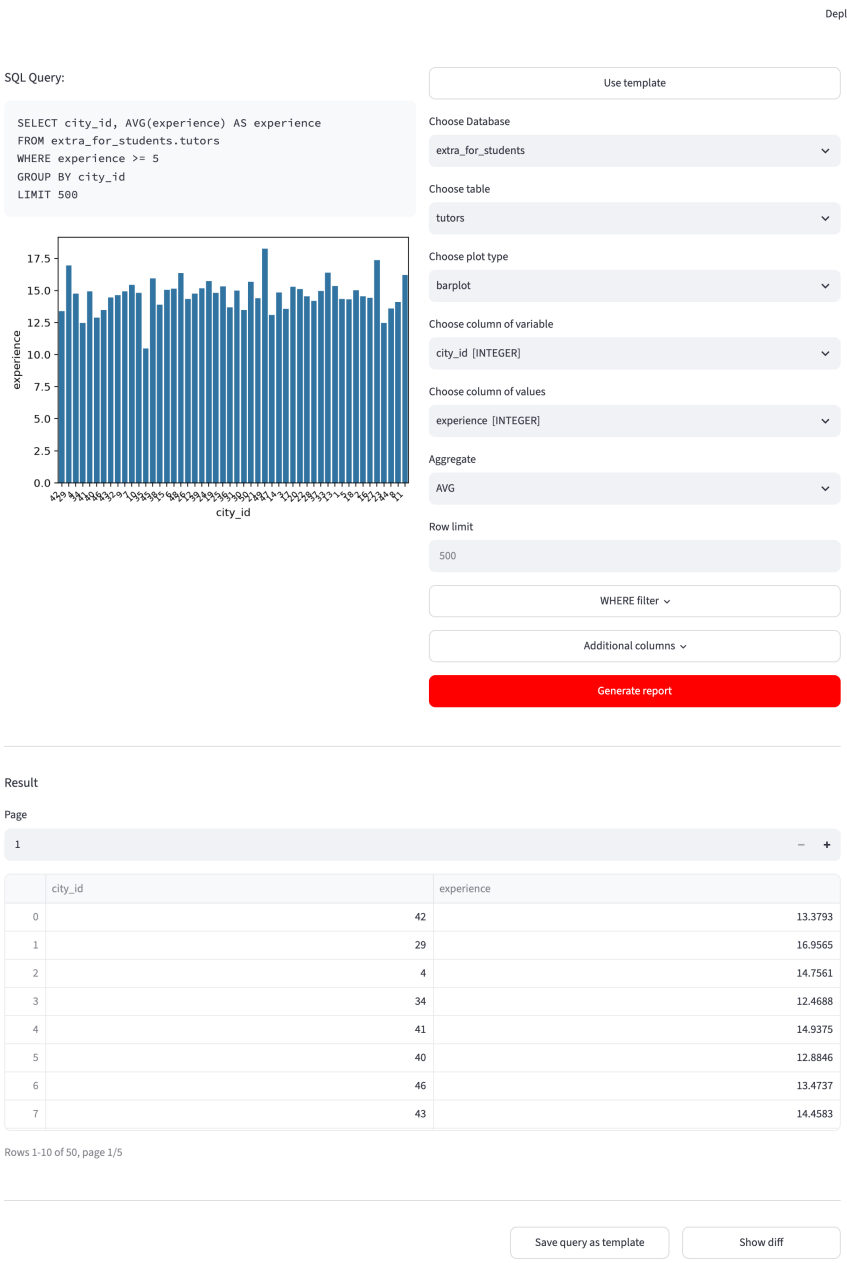



Рис. 31: Сценарий 8

## 10.9 Сценарий 9

Вывод суммарного числа спортсменов-призеров sportsman_count по занятым местам perfomance_place из таблицы perfomance базы sport_and_cheerleaders, где perfomance_place $\leq$ 3. Итоговый SQL запрос приведен в листинге 18, результат на рисунке 32.

```sql
SELECT perfomance_place, SUM(sportsman_count) AS sportsman_count
FROM sport_and_cheerleaders.perfomance
WHERE perfomance_place <= 3
GROUP BY perfomance_place
LIMIT 500
```

Листинг 18: Запрос сценария 9

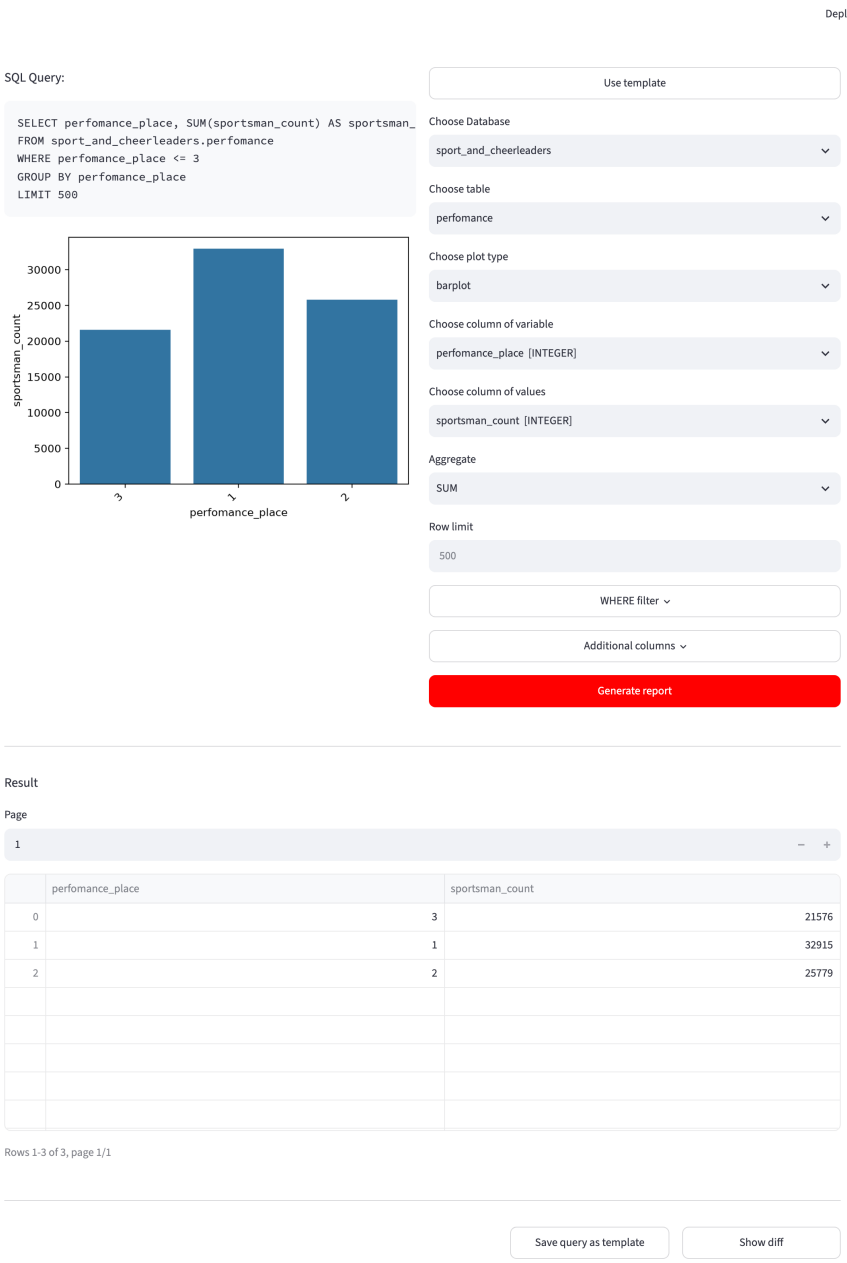



Рис. 32: Сценарий 9

## 10.10 Сценарий 10

Вывод количества результатов result_id по итогам выступления attempt_id (финишировал, DNF, DSQ, DNS) из таблицы result базы mountain_sport. Итоговый SQL запрос приведен в листинге 19, результат на рисунке 33.

```
SELECT attempt_id, COUNT(result_id) AS result_id
FROM mountain_sport.result
GROUP BY attempt_id
LIMIT 500
```

Листинг 19: Запрос сценария 10

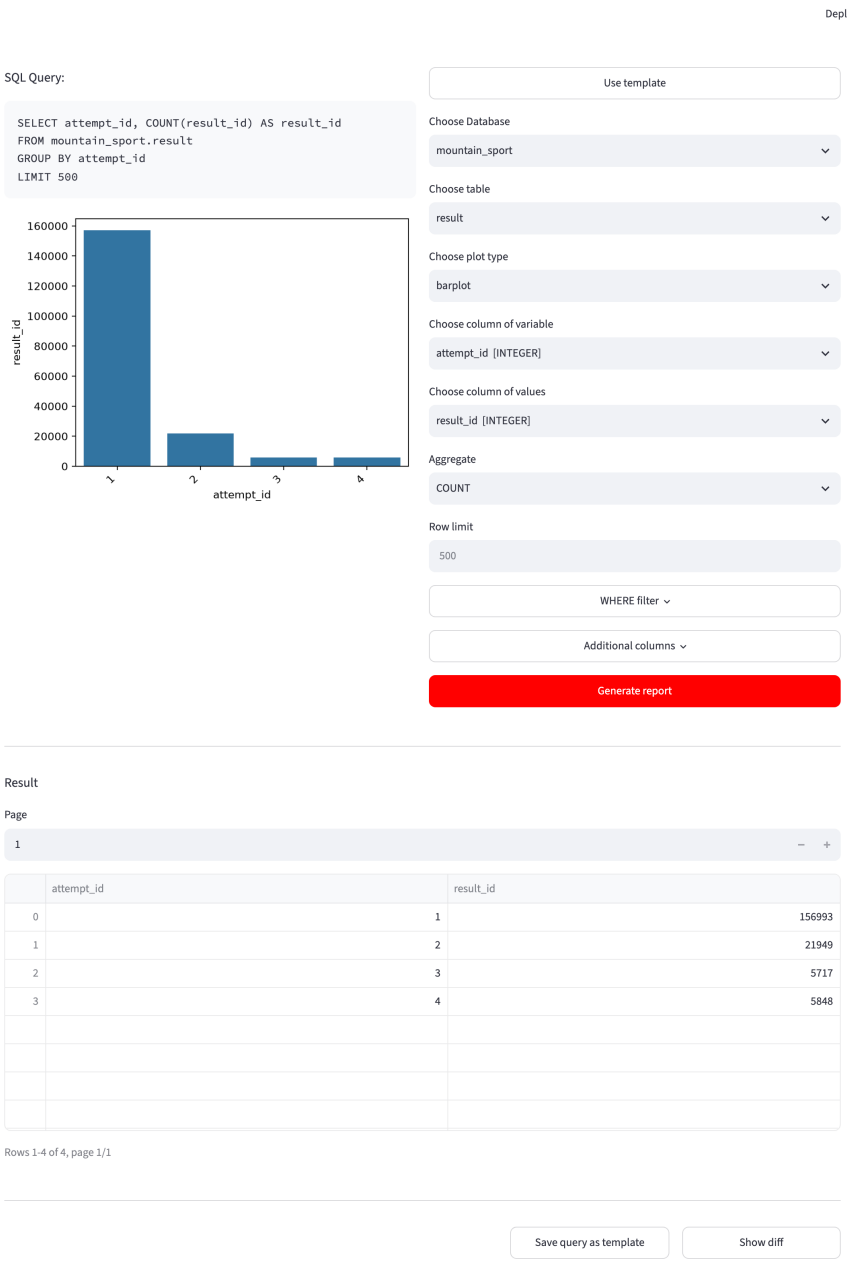



Рис. 33: Сценарий 10

## 10.11 Сценарий 11

Вывод среднего лучшего времени первой попытки attempt1_result по дисциплинам discipline из таблицы competition базы mountain_sport, где attempt1_result > 0. Итоговый SQL запрос приведен в листинге 20, результат на рисунке 34.

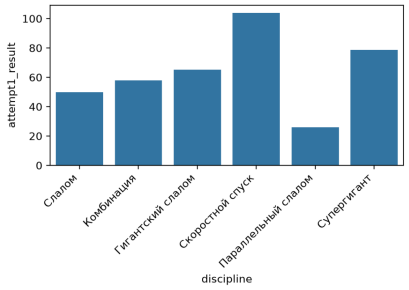
```
SELECT discipline, AVG(attempt1_result) AS attempt1_result
FROM mountain_sport.competition
WHERE attempt1_result > 0
GROUP BY discipline
LIMIT 500
```

Листинг 20: Запрос сценария 11

Deploy

SQL Query:

```
SELECT discipline, AVG(attempt1_result) AS attempt1_result
FROM mountain_sport.competition
WHERE attempt1_result > 0
GROUP BY discipline
LIMIT 500
```




Use template

Choose Database

mountain_sport

Choose table

competition

Choose plot type

barplot

Choose column of variable

discipline [VARCHAR]

Choose column of values

attempt1_result [NUMERIC]

Aggregate

AVG

Row limit

500

WHERE filter

Additional columns

Generate report

Result

Page

1

| | discipline | attempt1_result |
|---|---|---|
| 0 | Слалом | 49.8607 |
| 1 | Комбинация | 57.8396 |
| 2 | Гигантский слалом | 65.2006 |
| 3 | Скоростной спуск | 103.9427 |
| 4 | Параллельный слалом | 25.8131 |
| 5 | Супергигант | 78.7422 |

Rows 1-6 of 6, page 1/1

Save query as template

Show diff

Рис. 34: Сценарий 11

## Заключение

В ходе работы была спроектирована и реализована система, работающая с метаданными независимых баз данных в единой СУБД postgres. Развернуты три базы данных источников (учет репетиторов, горнолыжные соревнования, чирлидинг), заполненные сгенерированными данными, отдельная база метаданных с денормализованным аналогом информационной схемы meta.catalog и отдельная база шаблонов SQL запросов. Каждая база работает в собственном Docker контейнере со своим томом и изолированной сетью, а взаимодействие между базами выполняется через REST API.

Разработан интерфейс на Streamlit, в котором запрос к источнику собирается из форм по данным каталога метаданных: выбор базы, таблицы, колонок и агрегатной функции (SUM, AVG, COUNT), ограничение количества строк, вывод SQL запроса, таблиц результата и графика. Реализованы сохранение и повторное использование шаблонов запросов, сравнение двух шаблонов между собой, а так же просмотр отличий каталога от реальной схемы источника с возможностью обновления до текущего состояния.

Дополнительно к столбчатой диаграмме были реализованы круговая диаграмма (pieplot), диаграмма рассеивания (scatterplot) и тепловая карта (heatmap), форма условий WHERE с операторами сравнения для числовых колонок и поиском через LIKE для строковых, а так же вывод дополнительных колонок исходной таблицы.

В результате получилась минимальная BI система, позволяющая без написания SQL вручную выбирать данные из независимых источников, агрегировать и фильтровать их, визуализировать в виде графиков, сохранять запросы как шаблоны и отслеживать изменения структуры источников.

## Список источников


1. Chapter 35. The information schema. Postgresql [Электронный ресурс], URL: https://www.postgresql.org/docs/current/information-schema.html, дата обращения (23.09.2026).
2. Tableau Help. Tableau [Электронный ресурс], URL: https://www.tableau.com/support/help, дата обращения (25.09.2026).
3. PostgreSQL Metadata. IBM [Электронный ресурс], URL: https://www.ibm.com/docs/en/streamsets-legacy-dc/5.12.0?topic=processors-postgresql-metadata, дата обращения (23.09.2026).